

BUILDING LARGE LANGUAGE MODELS FROM SCRATCH

A Comprehensive Guide from Fundamentals to Production

Based on the YouTube Playlist by Andrej Karpathy et al.

Version 1.0

Table of Contents

Foreword

Chapter 1: Lecture 1: Building LLMs from scratch: Series introduction

Chapter 2: Lecture 2: Large Language Models (LLM) Basics

Chapter 3: Lecture 3: Pretraining LLMs vs Finetuning LLMs

Chapter 4: Lecture 4: What are transformers?

Chapter 5: Lecture 5: How does GPT-3 really work?

Glossary

Foreword

As we stand at the forefront of the artificial intelligence revolution, Large Language Models (LLMs) have emerged as a driving force behind the most impressive advancements in natural language processing. The ability to build and harness these complex systems has become a crucial aspect of modern AI research and development. However, despite their widespread adoption, the inner workings of LLMs remain shrouded in mystery for many. This is where "Building Large Language Models from Scratch" comes in – a comprehensive guide that delves into the fundamentals of LLMs, providing a thorough understanding of these powerful tools from the ground up.

Understanding LLMs from first principles is essential for anyone looking to contribute to the field or leverage these models in their own projects. By grasping the basic concepts and mechanisms that underlie LLMs, researchers and developers can unlock new possibilities for innovation and improvement. This book takes a rigorous and hands-on approach, walking readers through the process of building LLMs from scratch and exploring the key concepts that govern their behavior. From the basics of LLMs and the difference between pretraining and finetuning, to the inner workings of transformers and the architecture of state-of-the-art models like GPT-3, this book provides a detailed and comprehensive exploration of the subject matter.

The importance of understanding LLMs from first principles cannot be overstated. As these models become increasingly ubiquitous in our daily lives, it is crucial that we have a deep understanding of how they work, their strengths and limitations, and the potential risks and challenges associated with their use. By building a strong foundation in the fundamentals of LLMs, readers will be empowered to tackle complex problems, push the boundaries of what is possible, and contribute to the ongoing development of this exciting field. Whether you are a seasoned researcher or just starting out, this book provides a unique opportunity to gain a profound understanding of LLMs and their role in shaping the future of AI.

As you embark on this journey through the world of LLMs, you will discover the intricacies of these complex systems and the possibilities they hold. With its comprehensive coverage and detailed explanations, "Building Large Language Models from Scratch" is an invaluable resource for anyone looking to gain a deeper understanding of this rapidly evolving field. So, let us dive in and explore the fascinating world of LLMs, with a keen eye on the principles, concepts, and technologies that are driving this revolution forward. With this book as your guide, you will be well on your way to unlocking the full potential of LLMs and making your own mark on the future of AI.

Chapter 1

Lecture 1: Building LLMs from scratch: Series introduction

Building a large language model from scratch is a daunting task, but one that can be incredibly empowering. With the rise of generative AI and large language models transforming industries, the demand for experts who truly understand the underlying mechanics is growing. This series, led by Dr. Raj Dander, aims to fill this knowledge gap by providing a comprehensive, from-the-ground-up guide to building LLMs.

As we embark on the journey of building large language models from scratch, it's essential to understand the foundation and principles that govern these complex systems. The series of lectures that follows is designed to provide a comprehensive guide to constructing large language models, starting from the basics and working our way up to the most advanced concepts. The instructor, Dr. Raj Dander, introduces himself as a graduate of IIT Madras and MIT, with a PhD in machine learning, and outlines his mission to make AI accessible to everyone through the YouTube channel, VIUA. The channel has already created several playlists on machine learning and deep learning, all of which adhere to a philosophy of teaching from the ground up, without assuming prior knowledge.

This approach is particularly important in the field of large language models, where many learners make the mistake of diving straight into applications without understanding the underlying principles. They may run some code in a Google Colab notebook or follow a YouTube video, but few take the time to grasp the nuts and bolts of how these models actually work. The result is a lack of confidence and a superficial understanding of the subject matter. Dr. Dander's goal is to change this by providing a thorough and comprehensive guide to building large language models from scratch. By the end of this series, learners will have the knowledge and skills to construct their own large language model, and will find that subsequent applications and concepts seem much more accessible.

The philosophy behind this series is to teach everything from the basics, without assuming any prior knowledge. Dr. Dander emphasizes that this approach requires a huge amount of effort, but the payoff will be worth it. To support the video lectures, detailed lecture notes will be provided, and all the videos will be available for free. With this foundation in place, Dr. Dander begins to share his personal story of learning about large language models, and sets the stage for exploring the current state of available materials on the internet. He

notes that while there is a growing interest in large language models, with startups and companies increasingly adopting these technologies, the available resources often fall short of providing a thorough understanding of the subject. As we move forward in this series, we will delve into the specifics of what exists, what is lacking, and how this series aims to fill the gaps in our knowledge of large language models.

As we embark on the journey of building large language models from scratch, it's essential to understand the significance of this endeavor and the advancements that have been made in the field of natural language processing. To appreciate the progress, let's take a step back in time to the 1960s, when the field of natural language processing was still in its infancy. One of the earliest chatbots, called Eliza, was developed during this period. Eliza was designed to simulate a therapist, and its interface was straightforward. Upon choosing a language, Eliza would prompt the user to share their problem, and the conversation would unfold in a simplistic manner. For instance, if a user were to ask Eliza about resources for learning about large language models, the conversation would likely proceed in a series of vague and unhelpful responses, ultimately leading nowhere.

Fast forward to the present day, and we have chatbots like Chat GPT, which have revolutionized the way we interact with machines. If we were to ask Chat GPT the same question about learning resources for large language models, the response would be vastly different. Chat GPT would provide a plethora of useful information, including books, online courses, and research papers, demonstrating the tremendous progress that has been made in the field. This simple illustration highlights the remarkable advancements in natural language processing and large language models, making this an exciting time to be learning about and working with these technologies.

The fact that we can now build and interact with sophisticated large language models like Chat GPT is a testament to the power of research and development in this field. For those who may not be familiar with Chat GPT, don't worry – we will be building our own version of it throughout this series, and you will have the opportunity to learn about it in depth. The demonstration of Eliza and Chat GPT serves as a motivation to learn about large language models, and there are several other factors that make this an exciting time to be involved in this field. For instance, Facebook has recently released its Llama 3.1 model, which is one of its most capable large language models to date. What's notable about this release is that it is open-source, meaning that the entire architecture of the model is available for free, allowing anyone to access and learn from it.

In contrast to open-source models like Llama 3.1, many of the models released by other organizations, such as Open AI, are closed-source, meaning that the architecture and weights of the model are not publicly available. This distinction is significant, as it highlights the evolving landscape of large language models and the trade-offs between openness and secrecy. As we move forward in this series, we will explore the implications of these developments and delve into the technical details of building our own large language

models. The graph showing the trend of closed-source versus open-source models in 2022 and 2023 illustrates the shifting landscape of the field, with the release of models like GPT-4 in 2023, which, although closed-source, has had a profound impact on the world. As we continue to navigate this exciting field, it's clear that the future of large language models holds much promise and opportunity for innovation.

The field of large language models has experienced tremendous growth in recent years, with significant advancements in both open-source and closed-source models. When GPT-4 was released in 2023, it took the world by surprise, demonstrating impressive functionality despite being a closed-source model. However, the landscape has changed rapidly, and by 2024, the gap between open-source and closed-source models has begun to narrow. This shift is exemplified by the release of LLaMA 3.1, which performs on par with GPT-4, a closed-source model. This parity in performance underscores the notion that the information needed to build robust large language models is now readily available, and the primary barrier to entry is the willingness to learn.

For those unfamiliar with the distinction between open-source and closed-source models, or the specifics of LLaMA 3.1, fear not, as these topics will be explored in detail throughout this series of lectures. The realm of generative AI extends far beyond text, encompassing a broad range of capabilities, including video, audio, and 3D models. The applications of generative AI are vast and varied, and its potential is perhaps best illustrated by the stunning realism of AI-generated videos. These videos, which appear to be shot on location, are in fact created entirely by artificial intelligence. The sophistication and versatility of generative AI are a testament to the rapid progress being made in this field.

One area where the impact of large language models is particularly pronounced is in education. The development of AI-powered applications, such as the one created by the authors, has made it possible to generate high-quality educational content, including multiple-choice questions, in a matter of seconds. This represents a significant shift from traditional methods, which often relied on manual creation and curation of content. The potential applications of large language models in education are vast, and the authors' own experience in working with schools has highlighted the transformative potential of these technologies.

The growing importance of large language models and generative AI is also reflected in the job market, where demand for expertise in these areas is expected to grow exponentially in the coming years. As the field continues to evolve and mature, the need for skilled professionals who can develop and apply large language models will only increase. For those looking to learn about large language models, the question remains: where to start? A simple search online yields a plethora of courses and resources, but many of these focus on application development rather than the fundamentals of building large language models from scratch. It is this gap that the authors aim to address, providing a comprehensive and hands-on introduction to the world of large language models.

In this series of lectures, we will delve into the details of building large language models from scratch, exploring the theoretical foundations, practical applications, and real-world implications of these technologies. By the end of this journey, readers will have gained a deep understanding of the principles and practices underlying large language models, as well as the skills and knowledge needed to develop and apply these models in a variety of contexts. The next chapter will build on the foundations established here, exploring the technical and practical aspects of large language models in greater detail.

As we embark on the journey of building large language models from scratch, it's essential to recognize the existing gaps in educational resources that cater to this specific goal. Various courses, such as those focused on building LLM apps or mastering LLM concepts, often fall short of providing the in-depth foundations required to construct a large language model from the ground up. These courses typically prioritize app development or high-level concepts over the nuts and bolts of LLM construction, leaving learners without a comprehensive understanding of the underlying principles.

The desire for a detailed, foundational education in building LLMs from scratch is driven by the need to develop a robust skill set and confidence in tackling generative AI and LLM-related challenges, including job interviews. A quick crash course or superficial overview is insufficient for this purpose, as it fails to provide the depth and breadth of knowledge necessary for true mastery. A search for more in-depth resources on YouTube or other platforms often yields results that are similarly lacking, with many videos or playlists covering only a fraction of the required material and failing to provide a thorough, step-by-step guide to building an LLM from the very basics.

For instance, some courses or playlists may cover 18 chapters, but with each chapter lasting only 10-15 minutes, the content is often too concise and lacks the necessary technical depth. Other resources, such as Andre Karpaty's GPT course, may be too complex and assume prior knowledge, making them inaccessible to beginners. Even courses that claim to teach LLM construction from scratch often fall short, with explanations that are unclear or incomplete. This dearth of high-quality educational resources highlights the need for a comprehensive, technically oriented course that can provide learners with the solid foundation they require.

Fortunately, there are exceptions, such as the book by Sebastian Raschka, which provides an in-depth exploration of large language models. This book will serve as the foundation for our course, with its 48 pages of detailed information being converted into a series of video lectures. The goal is to create a resource that is both comprehensive and accessible, allowing learners to develop a deep understanding of the subject matter from the very basics. To achieve this, every aspect of the book will be meticulously covered, with detailed notes and explanations being presented in an engaging and easy-to-follow manner. By leveraging this book as a reference material and supplementing it with video content, we can create a learning experience that is both fundamental and interesting, providing

learners with the knowledge and skills they need to build large language models from scratch.

In the next lecture, we will begin to delve into the core concepts and principles outlined in the book, using detailed notes and explanations to provide a thorough understanding of the subject matter. The journey ahead will be guided by the goal of creating a comprehensive and accessible resource, one that will empower learners to build large language models from scratch and equip them with the skills and confidence they need to succeed in this exciting field.

As we embark on this comprehensive journey to build large language models from scratch, it is essential to start with a solid foundation, and that is exactly what this series aims to provide. The approach will be meticulous, with every detail written out and explained in a way that is both interesting and fundamental. The plan is to create a vast repository of knowledge, comprising lecture notes that could span 200 to 300 pages, accompanied by 50 to 60 videos that delve into each topic with clarity. This extensive resource will be designed to cater to serious learners who are eager to transition into the realm of large language models, ensuring that no prior knowledge is assumed and every concept is thoroughly elucidated.

The motivation behind creating this series is not just to provide a collection of videos, but to offer a structured learning path that equips students with the confidence to build large language models from the ground up. This is particularly important because, while tools like LangChain can facilitate the development of LLM applications, they do not replace the need for a deep understanding of how LLMs are constructed. Many learners may be tempted to start by deploying applications, often by cloning GitHub repositories, but this approach can lead to a superficial understanding of the subject matter. Without a thorough grasp of the fundamentals, individuals may find themselves struggling to answer detailed questions about key concepts such as query and values, positional embedding, or positional encoding.

The goal of this series is to empower learners with the knowledge and confidence to tackle complex questions and discussions about large language models. By mastering the skills to build an entire LLM from scratch, individuals will be better positioned to articulate their understanding of the subject, which is crucial in both academic and professional settings. For instance, in an interview, being able to discuss the intricacies of LLM architecture and training processes can make a significant difference in how one's expertise is perceived. This series aims to bridge the gap between theoretical knowledge and practical application, providing learners with a comprehensive understanding of LLMs that will serve as a solid foundation for their future endeavors.

In conclusion, this series is designed to provide a detailed and structured approach to learning about large language models, starting from the basics and progressing to

advanced topics. With a focus on building a strong foundation and equipping learners with the skills to build LLMs from scratch, this resource aims to fill a critical gap in the current landscape of LLM education. As we move forward, the next lecture will introduce the fundamentals of large language models, laying the groundwork for the in-depth exploration of LLMs that will follow. With this comprehensive series, learners will be well on their way to gaining a profound understanding of LLMs, setting them up for success in this rapidly evolving field.

Chapter 2

Lecture 2: Large Language Models (LLM) Basics

Building on the foundation established in the introductory lecture, we dive into the core concepts of large language models, exploring the fundamental aspects that define these powerful tools. Today's lecture will dissect the essence of large language models, clarifying key terms and concepts that are often shrouded in confusion. By examining the six major aspects of LLMs, we will gain a deeper understanding of their capabilities and applications.

As we embark on the journey of building large language models from scratch, it's essential to start with the fundamentals, and that's exactly what we'll be covering in this lecture. The previous lecture provided an introduction to the series, outlining the scope of topics we'll be exploring and our ultimate goal of gaining a deep understanding of large language models. To achieve this, we'll be delving into the inner workings of these complex systems, going beyond just their applications and exploring the underlying mechanics. Our exploration will be guided by the vision of making AI accessible to all, a mission that has driven my work for the past two years since completing my PhD at the Massachusetts Institute of Technology.

Today's lecture will be structured around six key aspects of large language models. We'll begin by defining what a large language model is, and then explore the significance of the term "large" in this context. We'll also examine the differences between modern large language models and their predecessors in the field of natural language processing. Furthermore, we'll investigate the secret ingredient that makes large language models so effective, and clarify the distinctions between various related terminologies, such as generative AI, deep learning, machine learning, and artificial intelligence. Finally, we'll discuss the exciting applications of large language models and the innovative projects that can be built using these technologies.

To lay the foundation for our exploration, let's start with the basics: what is a large language model? Unless you've been disconnected from the technological landscape, you've likely heard of large language models, perhaps in the context of Open AI or Generative AI. You may have even interacted with Chat GPT without fully understanding the underlying technology. At its core, a large language model is a neural network designed to understand, generate, and respond to human-like text. This definition comprises two key components: the neural network itself and the specific task of

processing human-like text. A neural network, in simple terms, is a series of layers of neurons that process input data, with the output layer generating the final result. The name "neural network" is derived from the symbolic representation of the brain's circuitry, with deep neural networks demonstrating an impressive capacity for complex tasks.

The concept of a neural network is fundamental to our understanding of large language models. Essentially, a neural network is a hierarchical structure, where input data flows through multiple layers of interconnected neurons, ultimately producing an output. This architecture is inspired by the human brain's neural structure, albeit in a highly abstract and simplified form. The term "deep" in deep neural networks refers to the stacking of multiple layers, which enables these models to learn and represent complex patterns in data. As we delve deeper into the world of large language models, it's crucial to grasp the basics of neural networks, and we'll be exploring these concepts in more detail throughout the series.

As we delve into the world of large language models, it's essential to understand the fundamental concepts that underpin these complex systems. Deep neural networks, which are the backbone of large language models, are composed of neurons that represent the circuitry in our brain, albeit in a highly symbolic sense. This is why they are referred to as deep neural networks, and their applications are vast and varied, including image detection, text generation, self-driving cars, and even medical diagnosis, such as detecting brain tumors. Large language models, in particular, are designed for generic text-related applications, including understanding, generating, and responding to human-like text.

But what does it mean to understand, generate, and respond to human-like text? A simple yet effective way to demonstrate this concept is to interact with a chatbot like Chat GPT. By engaging with Chat GPT, we can see how it understands and responds to human-like text in a remarkably human-like way. For instance, when asked to plan a relaxing day, Chat GPT responds by asking about the user's favorite forms of relaxation, such as reading, spending time outdoors, or engaging in creative pursuits. The conversation that ensues is eerily similar to one we might have with a human, with Chat GPT providing a personalized plan for a rejuvenating day, complete with suggestions for morning meditation, healthy breakfast, reading time, and a nature walk.

This ability to understand, generate, and respond to human-like text is at the core of what makes large language models so powerful. However, it's essential to remember that, despite their impressive capabilities, large language models are, at their core, simply deep neural networks trained on massive amounts of data. They are designed to perform specific tasks, such as understanding and generating human-like text, and they do so with remarkable accuracy. As we continue to develop and refine these models, we are seeing significant improvements in their ability to respond like humans, which has led to both exciting opportunities and challenging problems. The fact that large language models can now respond in a way that is almost indistinguishable from human communication has

raised important questions about their potential impact on society, and it is likely that we will need to devote significant attention to addressing these issues in the future.

The term "large language model" may seem straightforward, but it begs the question: why not simply call it a language model? The distinction is important, and it is a topic that we will explore in more detail as we continue our discussion of large language models. For now, it is sufficient to say that the term "large" refers to the scale and complexity of these models, which are capable of processing and generating vast amounts of human-like text. As we move forward in our exploration of large language models, we will examine the implications of this scale and complexity, and we will delve deeper into the technical details that underpin these remarkable systems.

As we delve into the world of large language models, a fundamental question arises: what exactly is a large language model, and why is it distinguished from a standard language model? The term "large" is not merely a colloquialism; it refers to the significant difference in model size, specifically the number of parameters, that characterizes these models. Unlike their predecessors, large language models boast an enormous number of parameters, often in the billions or even trillions. To put this into perspective, consider the evolution of the GPT model: GPT-3 Small had 125 million parameters, while GPT-3 Large had 760 million, and GPT-3 175B had a staggering 175 billion parameters. This exponential growth in parameters is a key factor in the advancements achieved by large language models.

The magnitude of these numbers can be difficult to comprehend, but a visual representation can help illustrate the point. A table comparing the number of parameters in different GPT models reveals the rapid escalation in model size. The "b" in GPT-3 175B, for instance, denotes the number of parameters in billions, underscoring the massive scale of these models. Similarly, a graph showing the comparison between GPT-1, GPT-2, and GPT-3 demonstrates not only the improvement in performance but also the tremendous increase in the number of parameters. The jump from GPT-1 to GPT-2, for example, represents a factor of 10 increase in parameters, while the transition from GPT-2 to GPT-3 is a factor of 100.

Other metrics, such as the number of decoders and token size, also exhibit a trend of growth. GPT-1 had 12 decoders, GPT-2 had 48, and GPT-3 had 96, while token size increased from 512 in GPT-1 to 1024 in GPT-2 and 2048 in GPT-3. Although the significance of these terms may not be immediately clear, they will be explored in more detail throughout this series. The broader context of model size in artificial intelligence is also noteworthy. A graph published in *Nature*, spanning from 1950 to 2022, shows the logarithmic increase in the number of parameters over the years. Until the 2020s, models rarely exceeded 100,000 parameters, but with the advent of large language models, the number of parameters has soared to 100 million, 1 billion, and even approached 1 trillion.

The graph highlights the remarkable progress in model size, particularly in the realm of language tasks, as represented by the orange symbols. The concentration of these symbols in the upper right corner of the graph underscores the dominance of large language models in pushing the boundaries of model size. As we continue to explore the intricacies of large language models, it is essential to appreciate the historical context and the unprecedented scale of these models. In the next section, we will delve deeper into the architectural and technological advancements that have enabled the development of large language models, setting the stage for a more comprehensive understanding of their capabilities and applications.

As we delve into the world of Large Language Models, it's essential to understand the significance of their massive scale. Models with 100 million, 1 billion, and even almost 1 trillion parameters have become the norm, with the majority of these parameters dedicated to language tasks. The sheer size of these models has led to them being dubbed "Large" Language Models, a term that reflects the unprecedented number of parameters they possess. This scale is a hallmark of the current era, where models have grown to an enormous size, warranting the "large" designation.

The term "Large Language Models" is twofold, with the second part, "Language Models," being equally important. This refers to the fact that these models are designed specifically for language-related tasks, excluding other modalities like images or videos. They are capable of performing a wide range of natural language processing tasks, including question answering, translation, sentiment analysis, and many more. The versatility of these models in handling various language-related tasks is a key reason why they are referred to as "Language Models." For instance, models like chat GPT have demonstrated their ability to perform a broad range of tasks, from simple text completion to complex language translation, and even drafting emails based on custom instructions.

The rise of Large Language Models has marked a significant shift in the field of natural language processing. While NLP has been around for a long time, the models that existed before the advent of Large Language Models were typically designed for specific tasks. For example, there were models dedicated solely to language translation or sentiment analysis. In contrast, Large Language Models are capable of performing a wide range of NLP tasks, making them more versatile and generic. This is evident in the fact that a model like GPT, which was initially trained for text completion, can also be used for language translation and other tasks. The flexibility and broad applicability of Large Language Models have set them apart from their predecessors and contributed to their popularity.

The differences between Large Language Models and earlier NLP models are not limited to their versatility. Earlier models struggled to perform tasks that required a deeper understanding of language, such as drafting emails based on custom instructions. Modern Large Language Models, on the other hand, can handle such tasks with ease. For instance, chat GPT can draft an email to book movie tickets based on a set of instructions,

a task that was previously challenging for earlier language models. This significant improvement in performance and capabilities has contributed to the widespread adoption of Large Language Models. As we continue to explore the world of Large Language Models, we will delve deeper into their architecture, training methods, and applications, gaining a better understanding of what makes them so powerful and versatile.

The ability of large language models to perform a wide range of tasks has made them incredibly versatile and powerful tools. A simple example of this can be seen in the task of drafting an email to a colleague or friend to book movie tickets. While this may seem like a trivial task, it is actually a great illustration of the capabilities of large language models. Not only can they generate text, but they can also insert emojis and other elements to make the email more engaging and personalized. This is a far cry from earlier natural language processing models, which struggled to perform such tasks. The key difference between large language models and their predecessors is their breadth of capabilities and the endless possibilities for their application.

One of the main reasons why large language models have been able to achieve such impressive results is that they are capable of learning from vast amounts of data and generalizing to new tasks. This has led to a wide range of applications, from language translation to text summarization, and has made large language models an essential tool for many industries. However, the question remains: what makes large language models so good at what they do? Is there a secret sauce that sets them apart from other natural language processing models? The answer to this question lies in the Transformer architecture, which is the foundation upon which most large language models are built.

For those who may be unfamiliar with the term, the Transformer architecture has nothing to do with the popular movie franchise, but rather refers to a specific type of neural network architecture that is designed to handle sequential data, such as text. The Transformer architecture is based on a paper titled "Attention is All You Need," which was published in 2017 by a team of researchers from Google Brain. This paper introduced a new way of thinking about natural language processing, one that focused on attention mechanisms rather than traditional recurrent neural networks. The Transformer architecture is characterized by its use of self-attention mechanisms, which allow the model to weigh the importance of different parts of the input data when generating output. This is illustrated in a schematic diagram, which shows the various components of the Transformer architecture, including input embedding, multi-head attention, and output embedding.

At first glance, the Transformer architecture may seem complex and intimidating, with its many different components and technical terms. However, it is actually a relatively simple and elegant design, one that has been widely adopted in the field of natural language processing. The paper that introduced the Transformer architecture has been incredibly influential, with over 100,000 citations in just five years. This is a testament to the impact that this work has had on the field and the importance of the Transformer architecture in

the development of large language models. As we continue to explore the basics of large language models, we will delve deeper into the details of the Transformer architecture and how it works, providing a thorough understanding of the secret sauce that makes these models so powerful.

As we delve into the world of Large Language Models, it's essential to understand the underlying concepts and technologies that have revolutionized the field of artificial intelligence. A seminal paper, which has garnered over 100,000 citations in just five years, has been a game-changer in this domain. This 15-page paper, although dense and complex, has had a profound impact on the field, and its contents will be revisited throughout these lectures. The paper's significance lies in its introduction of revolutionary concepts, including positional encoding, dot product attention, and multi-head attention, which will be explored in subsequent lectures.

For those unfamiliar with the term "Transformer," there's no need to worry – we will be devoting lectures to breaking down these concepts, including the attention formula, key query value, and multi-head attention. The idea of a Transformer may seem daunting, but it's a crucial component of Large Language Models, and by the end of these lectures, its intricacies will be fully explained. The complexity of this paper is evident, with each page potentially requiring multiple videos to explain. However, by dedicating lectures to specific sections, we will unravel the mysteries of this groundbreaking paper.

Moving on to the fifth point of our discussion, it's essential to clarify the terminologies surrounding Large Language Models. The abundance of terms being thrown around, such as LLM, Deep Learning, Machine Learning, and Artificial Intelligence, can be overwhelming. To better understand these concepts, it's helpful to visualize them as a series of nested umbrellas. The broadest umbrella is Artificial Intelligence, which encompasses all other domains. Within AI lies Machine Learning, a smaller umbrella that contains Deep Learning, and finally, Large Language Models, the smallest and most specialized umbrella.

To illustrate the differences between these terms, consider the example of a chat assistant, such as Lanza's AI-powered assistant, Alisa. This chat assistant, which can understand and respond to user queries, falls under the umbrella of Artificial Intelligence. However, it's essential to distinguish between AI and Machine Learning. While AI refers to any machine that exhibits human-like intelligence, Machine Learning is a subset of AI that involves training machines to learn from data. Deep Learning, in turn, is a subset of Machine Learning that focuses on specific types of neural networks. Large Language Models, the subject of our lectures, reside within the Deep Learning umbrella, representing a specialized area of research that deals with the development of sophisticated language models. By understanding the relationships between these terms, we can better appreciate the complexities and nuances of Large Language Models.

As we continue to explore the world of Large Language Models, it's crucial to grasp these fundamental concepts and terminologies. In the subsequent lectures, we will delve deeper into the intricacies of Transformers, positional encoding, and attention mechanisms, providing a comprehensive understanding of the underlying technologies. By the end of this series, we will have covered the essential components of Large Language Models, providing a solid foundation for further exploration and research in this exciting field.

As we delve into the basics of Large Language Models, it's essential to understand the nuances of Artificial Intelligence, Machine Learning, and Deep Learning. To illustrate this, let's consider a simple chat assistant, such as the Lanza chat assistant. This chatbot presents a selection of topics to choose from, and upon selecting one, it responds with pre-programmed answers. For instance, if we choose the topic "flight cancelled," the chatbot is already programmed to provide alternatives and rebook options. This may seem like an example of AI in action, as it covers basic intelligent behavior. However, this is a rule-based chatbot, meaning it doesn't learn from user responses or adapt to individual user behavior. It will respond in the same way every time, without taking into account the user's specific needs or preferences.

This distinction is crucial, as it highlights the difference between AI and Machine Learning. While AI encompasses a broad range of intelligent systems, Machine Learning is a subset of AI that involves machines learning from data and adapting to new information. In other words, Machine Learning enables systems to learn from user interactions and improve their performance over time. The Lanza chat assistant, on the other hand, is an example of AI, but not Machine Learning, as it doesn't learn or adapt based on user inputs.

To further clarify the relationship between these concepts, let's consider the difference between Machine Learning and Deep Learning. Machine Learning involves a range of techniques, including neural networks, decision trees, and more. Deep Learning, however, is a subset of Machine Learning that specifically involves neural networks. For example, if we want to predict heart disease based on patient data, we might use a decision tree, which is a Machine Learning approach that doesn't involve neural networks. On the other hand, if we're using a convolutional neural network to detect objects in images, such as a coffee cup, we're using Deep Learning.

The key characteristic of Deep Learning is the use of neural networks to learn complex patterns in data. For instance, in handwritten digit classification, we can train a neural network to recognize digits based on a large dataset of examples. As the neural network learns from the data, it becomes increasingly accurate in recognizing new, unseen digits. This is a classic example of Deep Learning in action, where the neural network is used to learn complex patterns in data and make predictions or classifications.

In summary, we've explored the basics of Large Language Models, including the distinction between AI, Machine Learning, and Deep Learning. We've seen how rule-based chatbots,

like the Lanza chat assistant, can be considered AI, but not Machine Learning, and how Deep Learning is a subset of Machine Learning that involves neural networks. As we continue to build on these concepts, we'll delve deeper into the world of Large Language Models and explore the techniques and technologies that enable them to learn and generate human-like language.

As we delve into the basics of large language models, it's essential to understand the broader context of artificial intelligence, machine learning, and deep learning. To illustrate this, consider a neural network trained on a dataset of handwritten digits, where the task is to identify the digit when a new image is presented. This example showcases a neural network architecture with an input layer, two hidden layers, and an output layer, demonstrating a classic deep learning application. When tested, the neural network correctly predicts the digit, whether it's an eight or a seven, highlighting its ability to learn and understand patterns in the data.

This example is a subset of machine learning, which encompasses not only neural networks but also other techniques like decision trees. Deep learning, in turn, is a subset of machine learning that involves neural networks with multiple layers, capable of learning complex patterns in data, including images. However, large language models (LLMs) are a further subset of deep learning, distinguished by their focus on text data alone. This distinction is crucial, as LLMs do not involve images or other modalities, setting them apart from more general deep learning applications.

The relationships between these terms can be confusing, but understanding the hierarchy is essential. Artificial intelligence (AI) serves as the broadest umbrella, encompassing machine learning (ML), which in turn encompasses deep learning (DL). Within DL, we find LLMs, each representing a more specialized application. Additionally, when we combine LLMs with deep learning, we enter the realm of generative AI, which deals with creating new content across various media forms, including text, images, sound, and video. This blend of LLMs and deep learning enables generative AI to tackle a wide range of tasks, from generating human-like text to creating new images and videos.

The key to understanding these terms lies in recognizing their respective domains and applications. LLMs are specifically designed to process and generate human-like text, leveraging deep learning techniques to achieve this goal. In contrast, generative AI represents a more comprehensive approach, merging LLMs with deep learning to create new content across multiple modalities. By grasping these distinctions, we can better appreciate the unique strengths and capabilities of each field, from the broad umbrella of AI to the specialized applications of LLMs and generative AI.

As we conclude this exploration of the basics, it's clear that understanding the interplay between AI, ML, DL, LLMs, and generative AI is essential for building a strong foundation in large language models. In the next chapter, we will build on this foundation, diving

deeper into the specifics of LLM architecture and the techniques used to train these powerful models. With a solid grasp of the underlying concepts and relationships, we will be well-equipped to explore the intricacies of LLMs and their applications, ultimately unlocking the potential of these models to generate human-like text and tackle complex tasks in the realm of natural language processing.

Large Language Models (LLMs) are a subset of artificial intelligence that has been gaining significant attention in recent years, and understanding their applications is crucial to harnessing their potential. As we explore the vast possibilities of LLMs, it's essential to first clarify the differences between AI, ML, DL, LLM, and generative AI, and we hope that the explanations and visuals provided have been helpful in solidifying this understanding. For those who may still have doubts, we encourage you to reach out and ask questions, as we're committed to providing clarification and support.

One of the most exciting aspects of LLMs is their ability to create new content. By leveraging the capabilities of these models, we can generate unique and innovative content that may not exist anywhere else. For instance, if we were to ask an LLM to write a poem about the solar system in the format of a detective story, it could produce a piece like the one we see here: "In the quiet sprawl of the Milky Way, a detective roamed the Stars by Night and Day, his name was Orion in Cosmic Affairs, where Mysteries burst, the case on his desk, Dan quite bizarre." This poem, created on the fly, demonstrates the power of LLMs in generating new and original content. We can use LLMs to write books, generate media articles, and even produce entire stories, making them an invaluable tool for creatives and writers.

Another significant application of LLMs is in chatbots. These virtual assistants can interact with humans, answering questions and providing support, much like a human customer service representative. We've seen examples of this in action, where an LLM is used to engage in conversation, asking questions and responding to user input. This technology has the potential to revolutionize the way we interact with companies and services, and it's likely that in the near future, many customer service representatives will be AI-powered chatbots. In fact, banks, airlines, hotel reservation desks, and many other industries are already leveraging LLMs to build chatbots that can handle a wide range of tasks, from booking shows to helping customers open bank accounts.

The applications of LLMs don't stop there. Machine translation is another significant area where these models are making a significant impact. With the ability to translate text into any language, LLMs are breaking down language barriers and enabling global communication. For example, we can ask an LLM to translate a poem into French, and it can produce an accurate translation in a matter of seconds. This capability has the potential to replace traditional translation tools, such as Google Translate, and even support regional languages, albeit with some limitations.

As we continue to explore the vast potential of LLMs, it's clear that their applications are diverse and rapidly expanding. From content creation to chatbots and machine translation, these models are poised to revolutionize the way we interact with technology and each other. In the next chapter, we'll delve deeper into the technical aspects of building LLMs, providing a comprehensive guide for those looking to develop their own models and unlock the full potential of this exciting technology. By the end of this journey, you'll be equipped with the skills and knowledge to build your own LLM, opening up a world of possibilities for innovation and creativity.

Large language models, or LLMs, have a wide range of applications that make them incredibly versatile tools. One of the key applications of LLMs is translation tasks, which enable the quick translation of their output into any language. While support for regional languages is not yet comprehensive, it is an area of ongoing research, with many efforts focused on improving LLM outputs for regional languages. This is a significant development, as it has the potential to expand the reach and accessibility of LLMs in diverse linguistic contexts.

Another significant application of LLMs is new text generation, such as writing poems, books, and media articles. This capability allows LLMs to create new text that did not previously exist, which can be a powerful tool for content creation. Furthermore, LLMs can also be used for sentiment analysis, which involves identifying the sentiment or emotional tone of a given text. This can be particularly useful for hate speech detection on social media platforms, where the ability to quickly and accurately identify and flag problematic content is essential.

These applications, which also include text summarization, text rewriting, and conversation, are the foundation of what LLMs can do. To illustrate the potential of these applications, consider a portal recently created for school teachers, which utilizes LLMs to generate a range of educational materials, including lesson plans, multiple-choice questions, and worksheets. For example, a teacher can use the portal to create a lesson plan on a specific topic, such as gravity, aligned with a particular curriculum, and receive a comprehensive plan complete with objectives, assessment keys, and guided practice exercises. This can save teachers a significant amount of time and effort, and is a testament to the potential of LLMs to transform the way we approach education.

The capabilities of LLMs are not limited to these examples, however. They can also be used to generate questions and answers on a given topic, such as World War II, complete with correct answers and explanations. This is just one illustration of the many cool things that can be done with LLMs, and demonstrates the vast potential of these models to create innovative and useful applications. As we will see in later lectures, building such applications from scratch is within reach, and requires only a solid understanding of LLMs and their capabilities. Unfortunately, many students make the mistake of simply downloading code, running it, and making superficial changes, without truly grasping the

underlying principles and potential of LLMs. By taking the time to thoroughly understand LLMs, however, the possibilities are endless, and the potential for creativity and innovation is vast.

In conclusion, this chapter has provided an overview of the basics of large language models, including their applications and capabilities. We have seen how LLMs can be used for translation tasks, new text generation, sentiment analysis, and more, and have explored the potential of these models to create innovative and useful applications. As we move forward, we will delve deeper into the details of LLMs, exploring their architecture, training, and deployment, and examining the many ways in which they can be used to solve real-world problems. The next chapter will build on the foundations established here, providing a more in-depth look at the technical aspects of LLMs and their applications.

As we conclude our exploration of the basics of Large Language Models, it's essential to address a common pitfall that many students fall into. The fascination with the numerous applications of LLMs can be overwhelming, leading some to hastily download code, make superficial changes, and claim expertise in the field. However, this approach is misguided, as true understanding and mastery of LLMs can only be achieved by grasping the fundamental concepts that underlie these complex systems. The purpose of this foundational work is to provide a comprehensive understanding of the inner workings of LLMs, and it is this depth of knowledge that will enable students to make meaningful contributions to the field.

The applications of LLMs that we have touched upon are merely the tip of the iceberg, and the possibilities for innovation and exploration are vast. The current era is an exciting time for research and industrial applications, and the potential for growth and discovery is limitless. Nevertheless, it is crucial to recognize that superficial knowledge of LLMs will only take you so far. To make a genuine impact, it is necessary to delve deeper into the intricacies of these models, including the Transformer code, the mechanisms of key query value, and the role of positional encoding. Without a thorough understanding of these concepts, your knowledge will remain superficial, and you will be unable to fully harness the potential of LLMs.

As we bring this section to a close, it's worth reflecting on the importance of a thorough grasp of LLM fundamentals. The applications we have discussed are just a starting point, and it is up to students and practitioners to push the boundaries of what is possible. By focusing on the basics and developing a deep understanding of the underlying mechanisms, you will be well-equipped to tackle the challenges and opportunities that lie ahead. In the subsequent sections, we will delve deeper into the coding aspects of LLMs, providing a hands-on approach to building and working with these models. It is essential to ensure that you have a solid foundation in the basics before proceeding, so take the time to review, make notes, and ask questions to clarify any concepts that remain unclear.

In conclusion, this chapter has laid the groundwork for our exploration of Large Language Models, covering the essential concepts and principles that underlie these complex systems. We have seen how LLMs can be applied in various contexts, from research to industrial applications, and have emphasized the importance of developing a deep understanding of the underlying mechanisms. As we move forward, we will build on this foundation, diving deeper into the coding aspects of LLMs and exploring the stages of building these models. By the end of this journey, you will have gained a comprehensive understanding of LLMs, enabling you to contribute meaningfully to the field and unlock the full potential of these powerful tools. The next chapter will provide a detailed examination of the Transformer architecture, a crucial component of modern LLMs, and will mark the beginning of our hands-on journey into the world of LLM coding.

Chapter 3

Lecture 3: Pretraining LLMs vs Finetuning LLMs

Building large language models from scratch involves two crucial stages: pretraining and finetuning. Pretraining lays the foundation by training models on vast and diverse datasets, while finetuning hones their performance on specific tasks. This lecture delves into the intricacies of these stages, exploring the differences and applications of pretrained and finetuned LLMs.

As we delve into the intricacies of building large language models from scratch, it's essential to understand the various stages involved in this complex process. The journey of creating a large language model, or LLM, encompasses two primary stages: pre-training and fine-tuning. To appreciate the significance of these stages, it's crucial to recall the fundamentals of LLMs, which were explored in previous lectures. For those who may have missed the earlier discussions, a brief recap is in order: we examined the basics of LLMs, their evolution from earlier NLP models, and the key factors that contribute to their exceptional performance. We also clarified the differences between LLMs, general AI, deep learning, machine learning, and artificial intelligence, and touched upon the diverse applications of LLMs.

In the context of building LLMs, pre-training and fine-tuning are sequential processes that enable these models to acquire the necessary knowledge and adapt to specific tasks. To grasp the concept of pre-training, let's consider a scenario where we interact with a chatbot like ChatGPT. When we engage with such a model, we're often impressed by its ability to understand our questions, provide accurate responses, and even engage in conversation. But have we ever stopped to think about how these models acquire such remarkable capabilities? The answer lies in pre-training, which involves training the model on a vast and diverse dataset. This process enables the LLM to learn patterns, relationships, and structures within the data, ultimately allowing it to generate human-like responses.

The scale of pre-training is truly enormous, as exemplified by models like GPT-3, which boasted 175 billion parameters and was trained on a massive corpus of data. To put this into perspective, the original paper on GPT-3 has garnered a significant number of citations, and the dataset used for its training is a testament to the model's impressive capabilities. By training on such a large and diverse dataset, the LLM can develop a deep understanding of language, including its nuances, idioms, and context-dependent

expressions. This foundation is essential for the model's subsequent fine-tuning, where it will be adapted to specific tasks or domains. As we explore the intricacies of pre-training and fine-tuning, we will gain a deeper understanding of how LLMs are built from scratch and how they can be tailored to meet the demands of various applications.

As we delve into the realm of large language models, it's essential to understand the significance of pretraining and finetuning in their development. To illustrate this, let's take a look at the original paper on GPT-3, which is a prime example of a large language model trained on a massive dataset. The paper reveals that the model was trained on a staggering amount of data, including 410 billion words from Common Crawl, a vast and open repository of internet data. For the purposes of this discussion, we can approximate one token to be equal to one word, allowing us to grasp the enormity of the dataset. Common Crawl is a treasure trove of information, comprising a huge corpus of text data that has been scraped from the internet. By leveraging this dataset, the GPT-3 model was able to learn from a vast array of texts, including websites, books, and online forums.

The sheer scale of the dataset is awe-inspiring, with 410 billion words from Common Crawl, 20 billion words from Web Text 2, and 12 billion words from books. Additionally, the model was trained on 3 billion words from Wikipedia, bringing the total tally to approximately 300 billion tokens or words. This massive dataset is equivalent to an enormous library of texts, where the model can learn to recognize patterns, relationships, and context. The reason large language models like GPT-3 perform exceptionally well is that they are trained on such a huge amount of data, allowing them to learn and remember a vast range of concepts, much like a child learns and remembers information from their parents.

Initially, large language models were trained on tasks such as word completion, where the model is given a sentence with a missing word and is tasked with predicting the next word. For example, given the sentence "the lion is in the dash," the model would predict the next word, such as "forest." What's remarkable is that even though these models were trained on a simple task, they demonstrated an ability to perform a wide range of other tasks as well. This was a surprising discovery, and it has significant implications for the development of large language models. As OpenAI noted in a 2018 paper, "we also noticed that we can use the underlying language model to begin to perform tasks without even training on them." This capability is a testament to the power of pretraining, which enables models to learn generalizable representations that can be applied to a variety of tasks, including multiple-choice questions and more. By leveraging these pretrained models, we can unlock a wide range of applications and capabilities, which we will explore further in the next section.

Building on the foundation established in previous lectures, the underlying language model can be leveraged to perform a wide range of tasks, even without explicit training on those tasks. A notable example of this can be seen in the performance of language models on

tasks such as multiple choice questions, where the accuracy of the model increases as the language model itself improves. This is a significant finding, as it indicates that a language model trained on a single task, such as predicting the next word in a sequence, can also perform well on a variety of other tasks, including translation, answering multiple choice questions, summarization, sentiment detection, and more.

The reason for this versatility lies in the fact that the language model is trained on a massive amount of data, which enables it to develop a deep understanding of the underlying language structures and relationships. As a result, the model can apply this knowledge to a wide range of tasks, even if it was not specifically trained on those tasks. This is in stark contrast to traditional natural language processing approaches, where separate models are often trained for each specific task, such as language translation or question answering. In contrast, pre-trained language models offer a single, unified model that can perform a wide range of tasks, without the need for task-specific training.

To illustrate this concept, consider a hypothetical application where a pre-trained language model is used to generate multiple choice questions on a specific topic, such as Egyptian civilization. Even if the model was not specifically trained on this task, the pre-training process has equipped it with the necessary knowledge and understanding to generate high-quality questions. This is precisely what happens when interacting with language models like GPT, which can generate questions, answers, and even entire texts on a wide range of topics, without requiring explicit training on those tasks.

A practical example of this can be seen in a dashboard application that utilizes a pre-trained language model to generate multiple choice questions on a variety of topics, including photosynthesis. By providing input parameters such as the grade level and topic, the model can generate a set of questions that meet the specified requirements. This is achieved through an API call to the pre-trained model, which demonstrates the versatility and flexibility of these models. The pre-training process has enabled the model to develop a broad range of skills, allowing it to perform a wide range of tasks with ease.

The concept of pre-training is straightforward: it involves training a language model on a massive amount of data, allowing it to develop a deep understanding of the underlying language structures and relationships. This process enables the model to perform a wide range of tasks, without requiring explicit training on those tasks. However, as we will explore in the next section, there is a second stage to the process, known as fine-tuning, which is equally important for achieving optimal results. Fine-tuning is necessary because, while pre-trained models can perform a wide range of tasks, they may not always possess the specific knowledge or expertise required for a particular application. For instance, a pre-trained model may not have the same level of knowledge about a specific industry or domain, such as the aviation industry, as a model that has been fine-tuned on that specific task. This is where fine-tuning comes in, allowing the model to be adapted to a specific task or domain, and providing a more tailored solution for the user's needs.

As we explore the intricacies of building large language models, it's essential to understand the distinction between pretraining and finetuning, a crucial aspect of developing effective language models. The significance of finetuning lies in its ability to refine a pre-trained model to cater to specific tasks or domains, making it an indispensable tool for organizations seeking to develop customized language models. For instance, consider a scenario where an airline company wants to create a chatbot that can provide users with tailored responses to queries such as flight prices or schedules. If the company relies solely on a pre-trained model, the responses generated may be too generic, as the model has been trained on a vast amount of data that may not include the company's specific information.

In such cases, finetuning allows the company to take the pre-trained model and further train it on their own dataset, enabling the model to learn the nuances of the company's specific language and terminology. This process of refinement enables the model to generate responses that are tailored to the company's specific needs, making it a valuable tool for organizations seeking to develop high-quality language models. For example, a large educational company may want to use a language model to generate multiple-choice questions, but relying solely on a pre-trained model may not yield the desired level of quality. By finetuning the model on their own dataset, the company can create a model that is better suited to their specific application.

The primary purpose of finetuning is to refine the pre-trained model on a narrower dataset, allowing the model to learn the specific characteristics of the data and generate more accurate responses. This is particularly useful for organizations that have access to large amounts of proprietary data that are not publicly available. For instance, a bank like JP Morgan can take a pre-trained model and finetune it on their own dataset, enabling the model to generate responses that are specific to the bank's operations and terminology. Finetuning is an essential step for organizations that want to deploy language models in real-world applications, as it allows them to create customized models that can cater to their specific needs.

To illustrate the effectiveness of finetuning, consider the example of SK Telecom, a company that wanted to build a chatbot to improve customer service interactions for telecom-related conversations in Korean. By finetuning a pre-trained model on their own dataset, SK Telecom was able to create a customized model that could generate responses tailored to their specific requirements. This resulted in significant improvements in the chatbot's performance, demonstrating the value of finetuning in developing effective language models. As we continue to explore the capabilities of large language models, it's clear that finetuning plays a critical role in unlocking their full potential, and we will delve deeper into the specifics of this process in the subsequent sections.

The process of building a large language model, or LLM, involves two key stages: pretraining and finetuning. One of the primary advantages of finetuning is that it allows the

model to be tailored to a specific task or industry, resulting in significant improvements in performance. For example, SK Telecom was able to achieve a 35% increase in conversation summarization quality and a 33% increase in intent recognition accuracy by finetuning a pre-trained model, known as GP4, on its own training data. This finetuning process enables the model to learn from the company's specific data and adapt to its unique needs, resulting in a more effective and accurate tool.

Another example that illustrates the importance of finetuning is Harvey, an AI legal tool designed for attorneys. While a pre-trained model like GP4 may be strong at reasoning, it may lack the extensive knowledge of legal case history and other relevant information required for legal work. To build an effective AI tool for lawyers, it is necessary to include specific legal case history data and fine-tune the model further. This process of finetuning allows the model to learn from the unique data and requirements of the legal industry, resulting in a more useful and accurate tool. As can be seen from the example of Harvey, this approach has led to the development of a trusted legal AI platform that works with the world's best legal teams.

The need for finetuning is not limited to the legal industry, as evidenced by the example of JP Morgan Chase's AI-powered LLM suite. Despite the availability of pre-trained models like GP4, JP Morgan Chase chose to develop its own LLM suite, fine-tuning it with the company's own data to create a tool that is tailored to the specific needs of its employees. This approach highlights the importance of finetuning in creating effective and accurate LLMs, particularly in industries where sensitive or proprietary data is involved.

In fact, finetuning is a crucial step in building advanced LLM applications, particularly in production-level settings. While pre-trained models like GP4 may be sufficient for students or researchers, finetuning is necessary to create models that can meet the specific needs of industries or startups. Whether it is in the telecommunications sector, the legal sector, or the banking sector, finetuning allows LLMs to learn from unique data and adapt to specific requirements, resulting in more effective and accurate tools. As we continue to explore the process of building LLMs, it is clear that finetuning plays a critical role in unlocking the full potential of these powerful models.

In conclusion, the process of finetuning is essential in creating effective LLMs, allowing them to adapt to specific tasks, industries, or companies. By tailoring the model to the unique needs and data of a particular application, finetuning enables LLMs to achieve significant improvements in performance, resulting in more accurate and useful tools. This understanding of finetuning sets the stage for further exploration of LLM development, including the various techniques and considerations involved in creating effective and specialized models.

The process of building a large language model involves two distinct stages: pre-training and fine-tuning. For students and researchers, using a pre-trained model like GP4 can be

sufficient for initial explorations, but as applications become more advanced, fine-tuning is necessary to achieve specific goals. To clarify this concept, let's break down the two stages and explore the steps involved in each.

The first stage, pre-training, is a crucial step in which a large language model is trained on a massive dataset, often comprising billions or trillions of words. This dataset can include a wide range of texts, such as internet articles, books, media, research papers, and more. The importance of data in this stage cannot be overstated, as it provides the foundation for the model's language understanding capabilities. However, training a large language model on such a massive dataset comes with significant computational costs. The requirement for powerful GPUs and substantial computational energy can be a barrier for individuals without access to such resources. For instance, the pre-training cost for GPT3 was estimated to be around \$4.6 million, highlighting the enormous investment required for training a foundational model.

To illustrate the process more clearly, a pre-training plus fine-tuning schematic can be used to visualize the steps involved. The schematic begins with the data collection step, where a massive dataset is assembled. The second step involves training the large language model on this dataset, which requires significant computational power. This process can be thought of as an unsupervised learning task, where the model learns to recognize patterns and relationships within the data without prior labeling. The resulting pre-trained model, also known as a foundational model, possesses a wide range of capabilities, as demonstrated by models like GPT4, which can engage in conversation and respond to a variety of questions and prompts.

Once a pre-trained model is available, the next step is fine-tuning, where the model is adapted for specific applications. This involves training the pre-trained model on a labeled dataset, which allows the model to learn task-specific patterns and relationships. Fine-tuning enables the development of applications such as personal assistants, language translation bots, summarization tools, and classification models. By fine-tuning a pre-trained model on a specific dataset, companies, industries, or startups can create customized models tailored to their particular needs. The distinction between unlabeled data used in pre-training and labeled data used in fine-tuning is an important one, as it highlights the different learning paradigms employed in each stage.

In conclusion, the two-stage process of pre-training and fine-tuning is essential for building large language models. By understanding the importance of data, computational costs, and the distinction between pre-training and fine-tuning, developers can create powerful models that can be adapted for a wide range of applications. This foundation will be crucial as we move forward to explore more advanced topics in large language model development, including the design of specific architectures and the optimization of training procedures, which will be covered in the next chapter.

The distinction between pretraining and finetuning is a crucial concept in building large language models, and one that is often misunderstood. At its core, pretraining involves training a model on a vast amount of unlabeled data, typically in the form of unsupervised learning tasks such as auto-regression. In this context, auto-regression refers to the process of using a portion of a sentence to predict the next word, with the predicted word then being used as input for the next iteration of training. This process allows the model to learn the underlying patterns and structures of language without the need for explicit labeling.

In contrast, finetuning is typically performed on labeled data, where the model is trained to perform a specific task such as classification. For example, if we wanted to build a model that could classify emails as either spam or not spam, we would need to provide a labeled dataset where each email is explicitly marked as either spam or not spam. This labeled data serves as the basis for finetuning the model, allowing it to learn the specific patterns and relationships that are relevant to the task at hand. A good illustration of this is the case of legal data, where we might have a set of labeled case studies that provide the necessary context and information for the model to learn from.

The reason we distinguish between pretraining and finetuning is that they serve different purposes in the model-building process. Pretraining provides a foundation for the model, allowing it to develop a broad understanding of language and its underlying structures. Finetuning, on the other hand, allows us to adapt this foundation to a specific task or domain, providing the model with the necessary expertise to perform well in that context. This two-stage process is reflected in the terminology, with pretraining implying a preliminary or foundational stage that precedes the more specialized stage of finetuning.

It's worth noting that the terminology can sometimes be confusing, and it's not uncommon for students to wonder why we don't simply refer to the entire process as "training". However, the distinction between pretraining and finetuning is an important one, and understanding the difference between these two stages is crucial for working effectively with large language models. By recognizing the role of pretraining in establishing a foundational model, and the role of finetuning in adapting that model to a specific task, we can better appreciate the complexities of the model-building process and the importance of each stage.

As we move forward in our discussion of building large language models, it's helpful to keep in mind the schematic of pretraining followed by finetuning. This process is essential for creating models that are capable of performing a wide range of tasks, from text classification to language generation. By understanding the difference between pretraining and finetuning, we can better navigate the complexities of model development and create more effective models that are tailored to our specific needs. With this foundation in place, we can now turn to the practical steps involved in building a large language model, including the process of training on a large corpus of text data and creating an initially

pre-trained model. The first step in this process is to train on a large corpus of raw text data, which refers to regular text without any labeling information. This raw text provides the basis for the pretraining stage, allowing the model to develop a broad understanding of language and its underlying structures.

The development of a large language model (LLM) involves two crucial stages: pretraining and fine-tuning. Having established a solid foundation for LLMs, we now delve into the specifics of these two stages, which are essential for building a powerful and versatile model. The pretraining stage involves the creation of an initially pre-trained LLM, often referred to as a base or foundational model. For instance, models like GP4, discussed in a recent paper, demonstrate the capabilities of pre-trained LLMs in tasks such as text completion, sentiment analysis, and question answering. This highlights the fact that pre-training not only enables the model to perform specific tasks but also imbues it with a range of other capabilities, making it a robust foundation for further development.

The next step in the process is fine-tuning, where the pre-trained LLM is further trained on labeled data. Fine-tuning is essential when deploying a model in a production environment, as it allows the model to be trained specifically on a particular dataset. This stage has numerous applications across various sectors, including the legal, telecommunications, and finance industries, as seen in examples such as Telecom Harvey and JP Morgan Chase. Fine-tuning can be categorized into two primary types: instruction fine-tuning and fine-tuning for classification tasks. Instruction fine-tuning involves providing the model with labeled data in the form of instruction-answer pairs, enabling it to learn specific tasks such as language translation. For instance, a text translation company may use instruction fine-tuning to train a model to translate English into French, while an airline customer support system may use it to respond to customer inquiries.

In contrast, fine-tuning for classification tasks involves training the model on labeled data to classify inputs into predefined categories. This approach is often used in applications such as spam email detection, where the model is trained on a labeled dataset of emails to distinguish between spam and non-spam emails. A key distinction between pre-training and fine-tuning is the requirement for labeled data. Pre-training can be performed without labeled data, whereas fine-tuning typically necessitates a labeled dataset to achieve optimal results. This fundamental difference underscores the importance of understanding the specific requirements and applications of each stage in the development of an LLM.

As we conclude this lecture, we reflect on the two stages of building an LLM: pre-training and fine-tuning. Pre-training lays the foundation for the model's development, allowing it to learn from a vast corpus of diverse data and acquire a range of skills. Fine-tuning then enables the model to be specialized for specific tasks and applications, leveraging labeled data to achieve optimal performance. The distinction between instruction fine-tuning and fine-tuning for classification tasks highlights the versatility of LLMs and the importance of selecting the appropriate approach for a given application. With a solid understanding of

these two stages, we are well-equipped to move forward and explore more advanced topics in LLM development, which will be covered in the next chapter.

As we conclude our discussion on the two stages of building a large language model, it is clear that pre-training and fine-tuning are essential components of creating a robust and effective model. Pre-training, as we have seen, requires a massive corpus of diverse data and significant computational power, making it a costly endeavor. The expense of pre-training is evident in the estimated \$4.6 million spent on pre-training GPT-3, highlighting the need for substantial resources to undertake this step. Furthermore, access to high-performance GPUs is a crucial requirement for pre-training, as it enables the processing of vast amounts of data.

Once pre-training is complete, the next step is fine-tuning, which involves training the model on a labeled dataset to perform specific tasks such as classification, summarization, translation, and building custom chatbots. Fine-tuning is a critical step, as it allows companies to adapt the foundational model to their particular needs and create specialized models that can perform tasks with high accuracy. In fact, most companies that build their own language models will fine-tune them to achieve the desired level of performance, rather than relying solely on the pre-trained model. This approach not only enhances the model's capabilities but also increases the overall cost, as fine-tuning requires additional computational resources and expertise.

The importance of fine-tuning is evident in the fact that big companies never rely solely on foundational models. Instead, they invest in fine-tuning to create customized models that can meet their specific requirements. This approach has led to the development of specialized chatbots and other language-based applications that are tailored to specific industries or use cases. As we have seen, the process of building a large language model involves significant investment in both pre-training and fine-tuning, highlighting the need for substantial resources and expertise.

In conclusion, our discussion on pre-training and fine-tuning has highlighted the critical components of building a large language model. We have seen that pre-training requires significant computational power and resources, while fine-tuning enables companies to adapt the model to their specific needs. Throughout these lectures, we have aimed to provide a comprehensive overview of the key concepts and techniques involved in building large language models, using visual aids and examples to illustrate the concepts. As we move forward, we will delve into the technical details of Transformers and the "Attention is All You Need" paper, laying the foundation for our future discussions on coding and implementation. With a solid understanding of pre-training and fine-tuning, we are now well-equipped to explore the intricacies of Transformer architecture and its applications in natural language processing.

Chapter 4

Lecture 4: What are transformers?

Lecture 4 delves into the fundamental concept of Transformers, a crucial component in building large language models. This introductory exploration will uncover the history, significance, and basic workings of Transformers, while also clarifying their relationship with large language models. By examining the Transformer architecture, we will gain insight into key terminology such as embedding and tokenization.

The development of large language models has been revolutionized by a fundamental concept: the transformer. As we continue our journey of building large language models from scratch, it's essential to understand the role of transformers in this context. Having previously explored the two stages of building a large language model - pre-training and fine-tuning - we can now dive deeper into the architecture that underlies many modern large language models.

Pre-training involves training on a large, diverse dataset, while fine-tuning is a refinement process that involves training on a narrower dataset specific to a particular task or domain. Understanding these stages is crucial for appreciating the significance of transformers. For those who may not have seen the previous lecture, it's highly recommended to review the material to ensure a smooth transition between the different topics.

Today, we're excited to introduce the concept of transformers, although we won't be delving into the mathematical or coding details just yet. Instead, we'll focus on introducing the flavor of this concept, exploring its history, and understanding its relevance to large language models. We'll also examine the similarities and differences between large language models and transformers, as these terms are often used interchangeably. By the end of this lecture, we'll have a solid understanding of how transformers work, including the basics of key terminologies like embedding and tokenization.

The transformer architecture is a deep neural network architecture that was introduced in a seminal paper titled "Attention is All You Need" in 2017. This paper has had a profound impact on the field, with over 100,000 citations in just six to seven years. The GPT architecture, which is the foundation of chat GPT, originated from this paper and is heavily based on the transformer architecture proposed therein. Understanding the significance of this paper and the transformer architecture is crucial for appreciating the breakthroughs that have occurred in the field of large language models.

The paper "Attention is All You Need" revolutionized the field of natural language processing, and its impact can still be felt today. The fact that it has been cited over 100,000 times is a testament to its influence. As we explore the transformer architecture, we'll gain a deeper understanding of how it has enabled the development of large language models that can process and generate human-like language. By examining the schematic of how transformers work, we'll gain insight into the key components that make up this architecture and how they contribute to its overall functionality.

As we begin this journey into the world of transformers, it's essential to keep in mind that our goal is to understand the basics of this concept and its relevance to large language models. We'll take a high-level view of the transformer architecture, exploring its history, key components, and significance in the context of large language models. By doing so, we'll lay the foundation for a deeper understanding of how transformers work and how they've enabled the development of large language models that can generate human-like language.

The concept of transformers, a crucial component in building large language models, was first introduced in the paper "Attention Is All You Need." This seminal work laid the foundation for the development of transformer-based architectures, which have since become a cornerstone of natural language processing. To fully appreciate the significance of transformers, it's essential to understand the context in which they were proposed and the specific problems they were designed to solve.

The original paper, which we'll be referencing throughout this discussion, presents a neural network architecture that was initially designed for machine translation tasks, such as translating English to French or German. The authors proposed a novel mechanism that achieved significant advancements in these tasks, and their work has since been built upon to enable a wide range of other applications. Notably, the transformer architecture was not initially intended for text completion tasks, which are now a predominant use case for models like GPT. Instead, the focus was on developing an efficient and effective approach to machine translation.

To delve into the specifics of the transformer architecture, let's consider a simplified schematic of the model. This schematic, which we've adapted from a detailed version presented in the book "Building LLMs from Scratch" by Sebastian, provides a comprehensive overview of the transformer's components and their interactions. At a high level, the transformer architecture can be broken down into eight distinct steps, each of which plays a critical role in the overall functioning of the model. These steps, which we'll explore in detail, provide a clear intuition into the transformer's mechanics and are essential for understanding how it operates.

For the purposes of illustration, let's consider a specific example, such as converting English text into German. This example will serve as a useful reference point as we work

through the eight steps of the transformer architecture. By examining each step in sequence, we can gain a deeper understanding of how the transformer processes input data, generates representations, and ultimately produces output. This process, which may seem complex at first, can be broken down into a series of manageable components, each of which contributes to the transformer's remarkable capabilities.

As we proceed through the eight steps, it's essential to keep in mind that our goal is not to provide an exhaustive analysis of the transformer architecture in a single lecture. Rather, we aim to introduce the key concepts and provide a solid foundation for further exploration in subsequent lectures. The transformer architecture is a rich and complex topic, and a thorough understanding of its components and their interactions will require a significant amount of time and effort. Nonetheless, by starting with a clear understanding of the basics, we can establish a strong foundation for more advanced topics and eventually develop a deeper appreciation for the transformer's role in building large language models.

As we explore the intricacies of large language models, it's essential to understand the fundamental concepts that underlie their architecture, and one of the most critical components is the Transformer. To appreciate the significance of the Transformer, let's consider a concrete example: converting English text to German. This is the classic problem that the original Transformer architecture was designed to solve.

In this example, we start with an input text in English, which is then processed through a series of steps to produce the translated German text. The first step involves pre-processing the input text, a crucial stage that prepares the data for the subsequent transformations. Pre-processing is a multifaceted operation, but one of its primary functions is tokenization. Tokenization is the process of breaking down the input text into simpler units, or tokens, which can be thought of as individual words or subwords. This is necessary because the input text is often composed of complex sentences, which cannot be fed directly into the model.

When dealing with vast amounts of data, such as documents or Reddit posts, tokenization is essential to reduce the complexity of the input. For simplicity, we can imagine that each word is a single token, although in practice, this is not always the case. The tokenization process involves breaking down a sentence into its constituent words and assigning a unique identifier, or ID, to each word. For instance, the sentence "Fine tuning is Fun For All" would be broken down into individual words, and each word would be assigned a unique number. This process allows us to take a large corpus of text data, break it down into manageable tokens, and assign a numerical representation to each token.

The output of the tokenization process is then passed to the next stage, which is the encoder. The encoder is one of the most critical components of the Transformer architecture, and its primary function is to perform vector embedding. Vector embedding is a process that converts the numerical representation of each token into a vector that

captures the semantic meaning of the word. This is essential because the numerical IDs assigned during tokenization do not convey any information about the relationships between the words. By using vector embedding, the encoder can capture the nuances of language, such as the similarity between words like "dog" and "puppy".

The encoder's role in the Transformer architecture is to transform the input text into a representation that can be used by the subsequent stages of the model. This involves not only converting the input text into a numerical representation but also capturing the contextual relationships between the words. As we delve deeper into the Transformer architecture, we will explore the intricacies of the encoder and its role in enabling the model to learn complex patterns in language. For now, it's essential to understand that the encoder is a critical component that prepares the input text for the subsequent stages of processing, setting the stage for the model to learn and generate coherent text.

As we delve into the world of large language models, it becomes increasingly important to understand how to effectively represent the input data, and one crucial step in this process is encoding the semantic meaning between words. The tokenization method, where words are converted into numerical IDs, falls short in capturing the relationships between words. For instance, if we take the words "dog" and "puppy", tokenization would assign random IDs to each word, but it would not convey the fact that they are related to each other. This limitation highlights the need for a more sophisticated approach to represent the input data, an approach that can capture the semantic meaning between words.

This process of representing words in a way that captures their semantic meaning is called vector embedding. Vector embedding involves converting words into vectorized representations, where each word is represented as a vector in a high-dimensional space. This allows words with similar meanings to be closer together in the vector space, while words with dissimilar meanings are farther apart. For example, in a two-dimensional vector embedding, words like "King", "Man", and "Woman" would be clustered together, as they are related to each other. Similarly, words like "apple", "banana", and "orange" would be grouped together, as they are all fruits. This vectorized representation enables the model to capture the nuances of language and understand the relationships between words.

The process of creating these vector embeddings is a complex task that requires careful consideration. It is not as simple as randomly assigning vectors to words, as the resulting embeddings would not accurately capture the semantic meaning between words. Instead, neural networks are trained to learn the vector embeddings, a process that involves optimizing the placement of words in the vector space to reflect their semantic relationships. The goal of this process is to create a vector space where words with similar meanings are closer together, and words with dissimilar meanings are farther apart.

In the context of the Transformer architecture, the encoder plays a crucial role in converting the input text into vector embeddings. The encoder takes in the pre-processed

tokens and generates vector embeddings that capture the semantic meaning between words. This process involves generating vectors in a high-dimensional space, often with hundreds of thousands of dimensions, which allows the model to accurately capture the nuances of language. The resulting vector embeddings are then used as input to the rest of the model, where they are processed and transformed to produce the final output.

The use of vector embeddings has a significant impact on the performance of the model, as it allows the model to capture the subtle relationships between words and understand the context in which they are used. The ability to represent words in a way that captures their semantic meaning is a fundamental aspect of natural language processing, and it has far-reaching implications for the development of large language models. As we continue to explore the Transformer architecture, we will see how the vector embeddings are used to facilitate the processing of input text and generate coherent and contextually relevant output.

Transformers are a type of neural network architecture that has revolutionized the field of natural language processing, and at their core, they rely on the concept of embedding vectors to capture the semantic meaning between words. The process of creating these embedding vectors is a crucial step in the Transformer architecture, where input text is first converted into tokenized IDs, which are then transformed into a vectorized representation. This vectorized representation can be visualized as a three-dimensional space, where each word or token is mapped to a unique point in this space.

To illustrate this process, consider a simple example where we take a sentence and convert it into a sequence of tokenized IDs, which are then fed into an embedding model. The output of this model is a set of vectorized embeddings that capture the semantic relationships between the words in the input sentence. This is the first step in the Transformer architecture, and it can be thought of as the "left side" of the architecture, where the input sentences are converted into vector embeddings that capture their semantic meaning.

The next step is to feed these vector embeddings into the "right side" of the architecture, which is known as the decoder. The decoder takes the vector embeddings and uses them to generate the output text, one word at a time. For example, if we are translating a sentence from English to German, the decoder will use the vector embeddings to predict the first word of the translation, and then use the combination of the partial output text and the vector embeddings to predict the next word, and so on. This process continues until the entire output text has been generated.

A key feature of the Transformer architecture, as well as other architectures like GPT, is that the model generates one output word at a time, and has access to the partial output text at each step. This allows the model to use the context of the previous words to inform its predictions about the next word. For instance, if we are translating the sentence "This is

an example", the model will have access to the partial output text "Das ist ein" (the German translation of "This is an") as it predicts the next word, which is the translation of "example". The model will use this context, along with the vector embeddings, to generate the final translation.

The decoder is responsible for taking the vector embeddings and the partial output text, and using them to predict the next word in the output sequence. This is done by feeding the tokenized IDs of the partial output text into the decoder, along with the vector embeddings, and then using the output of the decoder to predict the next word. The final output of the model is then generated by passing the output of the decoder through an output layer, which produces the final translated text. In our example, the final translation would be the completed German sentence, which captures the semantic meaning of the original input sentence.

As we have seen, the Transformer architecture relies on the combination of embedding vectors and the decoder to generate output text, one word at a time. This process allows the model to capture the context and semantic meaning of the input text, and to use this information to generate accurate and coherent translations. In the next part of our discussion, we will dive deeper into the details of the Transformer architecture, and explore how the different components of the model work together to achieve this goal.

As we delve into the intricacies of large language models, it's essential to understand the fundamental components that drive their functionality, and one such crucial component is the Transformer. To illustrate the Transformer's operation, let's consider an example where the model is tasked with translating text from one language to another. In this case, the decoder produces the translated text one word at a time, with the final output being the complete translated sentence. The decoder's ability to generate the translated text is a result of its neural network architecture, which is trained using a loss function to minimize errors and optimize performance.

The process of translation involves several key steps, which are critical to the Transformer's operation. Initially, the input text is pre-processed by breaking it down into individual tokens, such as words or characters, and assigning a unique token ID to each one. These token IDs are then passed through the encoder, which converts them into vector embeddings. This vector embedding represents the tokens in a high-dimensional space, where the semantic relationships between words are preserved. The decoder then receives this vector embedding, along with the partial output text, and uses this information to predict the next word in the translated sentence.

To clarify this process, let's walk through the eight steps involved in the Transformer's operation. The first step is to receive the input text, which is then pre-processed and broken down into tokens. The second step is to assign token IDs to each token, which are then passed through the encoder. The encoder converts these token IDs into vector

embeddings, which capture the semantic relationships between the words. The decoder then receives this vector embedding, along with the partial output text, and uses this information to predict the next word in the translated sentence. This process continues until the final output is generated, with the decoder producing the translated text one word at a time.

The Transformer's architecture is more complex than this simplified explanation, with feed-forward layers and weights that need to be optimized during training. However, this simplified example illustrates the basic principles of how the Transformer operates, and it's essential to understand these principles before diving into the more complex aspects of the model. The Transformer's ability to capture semantic relationships between words is a key aspect of its functionality, and this is achieved through the use of vector embeddings. By projecting words into a high-dimensional space, the model can preserve the complex relationships between words and generate more accurate translations.

As we continue to explore the Transformer's architecture, it's essential to note that this explanation has not yet covered one of the most critical aspects of the model: attention. The paper that introduced the Transformer, "Attention Is All You Need," highlights the importance of attention mechanisms in the model's operation. However, to avoid overwhelming the reader, we've focused on providing a simplified explanation of the Transformer's basic principles. In the next section, we'll delve deeper into the attention mechanism and explore how it contributes to the Transformer's exceptional performance.

The Transformer architecture, as introduced in the seminal paper "Attention is All You Need," can seem intimidating at first glance, but it is essential to approach it with a simplified perspective to avoid unnecessary complexity. At its core, a Transformer is a neural network, and understanding it requires optimizing its parameters, just like any other neural network. Many students make the mistake of diving directly into the intricate details of the architecture, which can lead to frustration and fear of the subject. To avoid this, it is crucial to start with a simplified understanding of the Transformer, which is why we began with a basic representation of the architecture.

The Transformer architecture consists of two primary blocks: the encoder block and the decoder block. The encoder's primary function is to convert input text into embedding vectors, while the decoder generates output text from these embedding vectors and any partial output it has received. These two blocks are the foundation of the Transformer architecture, and it is essential to remember that they work together to achieve the desired outcome. It is worth noting that the GPT architecture, which was introduced later, differs from the original Transformer in that it lacks an encoder and relies solely on the decoder. However, for now, we will focus on the standard Transformer architecture, which includes both encoder and decoder blocks.

One of the key components of the Transformer architecture is the self-attention mechanism. This mechanism allows the model to weigh the importance of different words and tokens relative to each other, enabling it to capture dependencies between them, regardless of their distance in the input or output sequences. The self-attention mechanism is a type of attention mechanism that relates different positions of a single sequence to compute a representation of that sequence. This concept can be challenging to grasp, but it is essential to understand that the self-attention mechanism enables the model to assign importance to different words and tokens based on their context and relationships.

To illustrate this concept, consider a scenario where we have multiple sentences, such as "Harry Potter is on station or platform number something" and "Harry Potter wants to board the train." The self-attention mechanism allows the model to analyze the relationships between the words and tokens in these sentences, assigning importance to each word based on its context and relevance to the other words. This enables the model to capture complex dependencies and relationships between the input sequences, which is critical for tasks such as language translation and text generation.

As we delve deeper into the Transformer architecture, it becomes clear that the self-attention mechanism is a crucial component that enables the model to capture complex dependencies and relationships between input sequences. In the next part of our discussion, we will explore the self-attention mechanism in more detail, examining how it works and how it is implemented in the Transformer architecture. By understanding the self-attention mechanism, we can gain a deeper appreciation for the power and flexibility of the Transformer architecture, which has revolutionized the field of natural language processing.

As we delve into the world of transformers, it's essential to understand the concept of long range dependencies, which plays a crucial role in the architecture of these models. Consider a scenario where we have a series of sentences, such as "Harry Potter is on station or platform number something," followed by "Harry Potter wants to board the train," and then a third sentence that sets the stage for a fourth sentence. When predicting the next word in the fourth sentence, the context provided by the previous sentences is vital. The model needs to know what was mentioned in the first, second, and third sentences to accurately predict the next word in the fourth sentence. This is where the concept of long range dependencies comes into play, as the model must capture the relationships between words that are far apart in the input sequence.

The self-attention mechanism is a key component that enables transformers to capture these long range dependencies. This mechanism allows the model to weigh the importance of different words or tokens relative to each other, effectively giving more attention to certain words when predicting the next word in a sequence. For instance, in our example, the model may give more attention to the word "Harry" or "train" when predicting the next word, as these words are more relevant to the context. The

self-attention mechanism maintains an attention score, which is a matrix that tells the model which words to prioritize when making predictions. This score is calculated based on the relationships between the input words, allowing the model to focus on the most relevant information.

The architecture of transformers reflects the importance of attention, with attention blocks that are designed to capture long range dependencies. These blocks, also known as multi-head attention, enable the model to look at the input sequence from different perspectives, allowing it to capture a wider range of relationships between words. The paper that introduced transformers, "Attention is All You Need," emphasizes the importance of self-attention in enabling models to capture long range dependencies. By calculating an attention score, the model can determine which words to prioritize, allowing it to make more accurate predictions.

The intuition behind attention is simple yet powerful. As humans, we naturally keep context in mind when reading a story, remembering what was written on previous pages. However, for a model to do the same is much more challenging. The self-attention mechanism allows models to capture long range dependencies, enabling them to make accurate predictions. This is particularly evident in models like chat GPT, which use attention to focus on every sentence in the input sequence before predicting the next word. By giving attention to every sentence, the model can capture the relationships between words and make more informed predictions. As we explore the mathematics and coding behind attention, it's essential to appreciate the beauty of this mechanism and how it enables models to capture the complex relationships between words in a sequence.

In the context of transformers, the self-attention mechanism is a game-changer, allowing models to capture long range dependencies and make more accurate predictions. As we continue to explore the architecture of transformers, it's essential to keep in mind the importance of attention and how it enables models to focus on the most relevant information. By understanding how attention works, we can appreciate the complexity and beauty of transformer models, and how they have revolutionized the field of natural language processing. With this foundation in place, we can move on to explore the mathematical and coding aspects of attention, and see how it is implemented in transformer models.

Transformers have revolutionized the field of natural language processing, and one of the key mechanisms that enables their impressive performance is the self-attention mechanism. This mechanism allows models like chat GPT to accurately predict the next word in a sentence by giving attention to every sentence, not just the previous one. In other words, when you input a sentence into GPT, it doesn't just look at the immediate context, but rather considers the entire sequence of words to determine the most likely next word. This is particularly useful when previous words are more important for understanding the context, and it's all made possible by the self-attention mechanism.

This concept is crucial to understanding how transformers work, and it's one of the key ideas that enables models like GPT to achieve state-of-the-art results. The Transformer architecture, which was introduced in a 2017 paper, has since been extended and modified to create new models like BERT and GPT. BERT, which stands for Bidirectional Encoder Representations from Transformers, is a model that predicts hidden words in a given sentence. It does this by randomly masking some words in the input sentence and then trying to predict the missing words. This approach is different from GPT, which generates new words based on the input sequence.

To illustrate how BERT works, consider a sentence like "This is an example of how LLM can _____". During training, BERT would receive this sentence with some words randomly masked, and it would try to predict the missing words. The input sentence would be pre-processed, converted into token IDs, and then passed through the encoder, which would generate embeddings for each word. The output would be the predicted missing words, which in this case might be "perform". This process allows BERT to learn a deep understanding of language and how words relate to each other.

In contrast, GPT works by predicting the next word in a sequence, given an incomplete sentence. For example, if the input sentence is "This is an example of how LLM can", GPT would try to predict the next word, such as "perform". The pre-processing steps are similar to BERT, but the goal is different. GPT is designed to generate coherent and natural-sounding text, one word at a time. By understanding how these two models work, we can appreciate the flexibility and power of the Transformer architecture, which has enabled a wide range of natural language processing applications.

As we explore the Transformer architecture and its variants, it's clear that there are many different approaches to natural language processing, each with its own strengths and weaknesses. In the next part of this chapter, we will delve deeper into the details of the Transformer architecture and its applications, and explore how these models are being used in real-world scenarios. By examining the inner workings of these models, we can gain a deeper understanding of how they achieve their impressive results, and how we can apply these techniques to our own natural language processing tasks.

Transformers have revolutionized the field of natural language processing, and two of the most popular transformer-based models are GPT and BERT. To understand how these models work, let's first consider the task of predicting the next word in a sentence. Given incomplete text, the goal is to predict the next word, which is the core functionality of GPT. The way GPT works is by performing pre-processing, converting words to token IDs, and then using a decoder model to predict the last word or the word that is not known. This process allows GPT to generate one word at a time, a capability that enables it to perform a wide range of tasks, including completing missing text and sentiment analysis.

One of the key differences between GPT and BERT is the direction in which they process text. GPT models are designed to process text from left to right, meaning that they only have access to the context that comes before the word they are trying to predict. In contrast, BERT models are designed to process text in both directions, meaning that they have access to the context that comes both before and after the word they are trying to predict. This allows BERT to capture nuances and relationships between words that are important for understanding meaning and context. For example, a BERT model can differentiate between "bank" as a financial institution and "bank" as a riverbank by looking at the surrounding words.

The ability of BERT to process text in both directions is reflected in its name, which stands for Bidirectional Encoder Representations from Transformers. This bidirectional processing allows BERT to understand the relationships between words in a sentence, making it particularly well-suited for tasks such as sentiment analysis. While GPT models can also be used for sentiment analysis, BERT's ability to capture the nuances of language makes it a more popular choice for this task. It's worth noting that both BERT and GPT have originated from the transformer architecture, which is a key technology that has enabled the development of these powerful language models.

As we explore the differences between BERT and GPT, it's also important to consider the distinction between these models and the broader category of large language models (LLMs). While BERT and GPT are both LLMs, not all LLMs are necessarily transformers. The term "transformer" refers specifically to the architecture that underlies these models, which is designed to handle sequences of data, such as text or images. Understanding the relationship between transformers and LLMs is essential for appreciating the capabilities and limitations of these models, and for designing new models that can take advantage of the strengths of both approaches.

In conclusion, this chapter has provided an overview of the key differences between BERT and GPT, two popular transformer-based models. We have seen how these models process text in different directions, and how this affects their ability to capture nuances and relationships between words. We have also touched on the relationship between transformers and LLMs, and the importance of understanding this distinction for designing and working with these models. In the next chapter, we will delve deeper into the transformer architecture, exploring its key components and how they work together to enable the powerful capabilities of models like BERT and GPT.

As we delve into the world of large language models, it's essential to understand the relationship between Transformers and LLMs, and to clarify the differences between these two concepts. One of the key distinctions to note is that not all Transformers are LLMs. While Transformers have become synonymous with language tasks, they can also be applied to other domains, such as computer vision. In fact, Transformers have been used for image recognition and classification, among other vision tasks, through the

development of Vision Transformers, also known as ViT. These architectures have shown remarkable results, often outperforming traditional convolutional neural networks (CNNs) while requiring substantially fewer computational resources for pre-training.

The applications of Vision Transformers are diverse and significant, ranging from detecting potholes on roads to classifying tumors as malignant or benign from medical images. The similarities and differences between CNNs and Vision Transformers have been a topic of discussion, with the latter demonstrating a weaker bias and improved performance in certain tasks. This highlights the importance of considering Transformers as a broader class of models, rather than solely in the context of language processing. The ability of Transformers to be applied to different domains, including computer vision, expands the scope of their potential applications and underscores their versatility.

However, it's equally important to note that not all LLMs are Transformers. LLMs can be based on other architectures, such as recurrent or convolutional neural networks. Recurrent neural networks (RNNs), for instance, were introduced in the 1980s and have been used for sequence modeling tasks, including text completion. Long short-term memory (LSTM) networks, which were developed later, incorporate two separate paths to handle short-term and long-term memories, making them particularly well-suited for tasks that require remembering context over time. These architectures, while distinct from Transformers, can also be classified as language models, highlighting the diversity of approaches within the field of LLMs.

To understand the role of RNNs and LSTMs in the context of LLMs, it's helpful to consider how they process information. RNNs maintain a feedback loop, allowing them to incorporate memory into their computations. LSTMs, on the other hand, use two separate paths to handle short-term and long-term memories, enabling them to capture a wider range of temporal dependencies. This distinction is crucial, as it highlights the different design choices and trade-offs that underlie various LLM architectures. By recognizing that LLMs encompass a broader range of models, including those based on RNNs, LSTMs, and Transformers, we can better appreciate the complexity and richness of the field, and the opportunities for continued innovation and research.

In conclusion, the distinction between Transformers and LLMs is not always clear-cut, and understanding the relationships between these concepts is essential for navigating the field of natural language processing. As we move forward, we will continue to explore the intricacies of Transformer architectures and their applications, as well as the broader landscape of LLMs and their role in shaping the future of artificial intelligence. The next chapter will build on these foundations, delving deeper into the technical details of Transformer models and their implementation, and providing a more comprehensive understanding of their capabilities and limitations.

As we delve into the world of large language models, it's essential to understand the nuances of transformers, a crucial component in many modern language models. To begin with, it's worth noting that not all transformers are large language models, and conversely, not all large language models are transformers. This distinction is vital, as the terms are often used interchangeably, despite being distinct concepts. In fact, various neural network architectures, including recurrent neural networks, long short-term memory networks, and even some convolutional architectures, can be used to build large language models. The key takeaway here is that while transformers are a popular choice for building large language models, they are not the only option, and understanding the differences between these concepts is crucial for working with language models.

To illustrate this point, consider a simple example. Imagine we have two paths, one for short-term memories and one for long-term memories. We can represent these paths as two lines, one green and one blue, where the green line represents long-term memory and the blue line represents short-term memory. By using both paths, we can make predictions about what comes next, demonstrating how different components can work together to enable language modeling. This idea is not unique to transformers, as other architectures can also be used to build large language models. However, as we will see, transformers have become a popular choice due to their ability to capture long-range dependencies and weigh the importance of different words relative to each other.

The transformer architecture, as proposed in the 2017 paper "Attention is All You Need," is a deep neural network architecture that has become a cornerstone of modern language models. Originally developed for machine translation tasks, such as translating English text into German or French, the transformer architecture has been widely adopted for other language modeling tasks. At its core, the transformer architecture consists of an encoder and a decoder, although some variants, such as the GPT models, use only a decoder. The encoder takes in input text, converts it into vector embeddings, and passes these embeddings to the decoder, which generates output text one word at a time.

One of the key innovations of the transformer architecture is the self-attention mechanism, which allows the model to weigh the importance of different words relative to each other. This mechanism is what enables transformers to capture long-range dependencies and make accurate predictions about what comes next. By considering the context of the entire input sequence, the self-attention mechanism can identify which words are most relevant for predicting the next word, making it a powerful tool for language modeling. As we will explore in more detail in subsequent chapters, the self-attention mechanism is a critical component of the transformer architecture, and understanding how it works is essential for building effective language models.

In conclusion, this chapter has provided an overview of the transformer architecture and its role in building large language models. We have seen how transformers can be used to capture long-range dependencies and weigh the importance of different words relative to

each other, making them a powerful tool for language modeling. However, we have also emphasized the importance of distinguishing between transformers and large language models, as these terms are often used interchangeably despite being distinct concepts. As we move forward, we will delve deeper into the details of the transformer architecture, exploring its strengths and weaknesses, and examining how it can be used to build effective language models. The next chapter will build on this foundation, providing a more in-depth examination of the self-attention mechanism and its role in the transformer architecture.

As we delve into the nuances of transformer architectures, it becomes clear that one of the key benefits of these models is their ability to consider context from the entire input sequence when making predictions. This is achieved through self-attention mechanisms, which allow the model to weigh the importance of different words or tokens in the input sequence and determine how they relate to one another. This concept of self-attention can be thought of as a form of parallel attention to different parts of a paragraph or sentence, and it will be a crucial aspect of our exploration of transformers as we move forward.

One of the key variations of the transformer architecture is BERT, which stands for Bidirectional Encoder Representations. Unlike other transformer models, BERT is designed to predict hidden or missing words in a sentence, often referred to as masked words. To accomplish this, BERT uses a bidirectional approach, paying attention to the input sentence from both the left and right sides. This allows the model to capture the nuances of word relationships and contextual meaning, making it particularly well-suited for tasks like sentiment analysis. Notably, BERT relies solely on an encoder architecture, eschewing the decoder component found in other transformer models.

In contrast, models like GPT take a different approach, using a left-to-right architecture to predict the next word in a sequence. This involves receiving incomplete text and generating one word at a time, with the model relying on the context provided by the preceding words to inform its predictions. Unlike BERT, GPT consists only of a decoder component, with no encoder counterpart. This fundamental difference in design reflects the distinct goals and use cases of these two models, with BERT exceling at tasks that require a deep understanding of contextual relationships and GPT demonstrating strength in applications like text generation and completion.

It's essential to recognize that the terms "transformer" and "large language model" (LLM) are often used interchangeably, but this is not entirely accurate. While many LLMs are based on transformer architectures, the reverse is not necessarily true: transformers can be applied to a wide range of tasks beyond natural language processing, including computer vision applications like image classification and segmentation. Conversely, LLMs can be built using architectures other than transformers, such as recurrent neural networks or convolutional architectures. This distinction is crucial, as it highlights the unique strengths and limitations of each approach and underscores the importance of

understanding the underlying architecture and design choices.

As we conclude this lecture, we've covered a range of key points, from the basics of self-attention and transformer architecture to the distinct characteristics of models like BERT and GPT. We've also touched on the broader relationship between transformers and LLMs, highlighting the potential for confusion and the need for clarity. By taking the time to understand these concepts and distinctions, readers will be well-equipped to navigate the complexities of modern natural language processing and to appreciate the rich possibilities offered by these powerful models. With this foundation in place, we're ready to move forward, exploring the many applications and implications of transformers and LLMs in the chapters to come.

As we conclude our exploration of transformers, it's clear that these powerful models have revolutionized the field of natural language processing. The numerous examples and illustrations presented throughout this lecture have demonstrated the effectiveness of transformers in handling complex language tasks. The process of understanding and working with transformers is not only intellectually stimulating but also crucial for building large language models from scratch. With this foundation in place, we can now appreciate the significance of transformers in modern NLP applications.

In reflecting on the key takeaways from this lecture, it becomes evident that transformers have emerged as a vital component in the development of large language models. The ability to efficiently process sequential data, capture long-range dependencies, and parallelize computations has made transformers an indispensable tool for NLP practitioners. As we move forward in our journey to build large language models, the concepts and techniques introduced in this lecture will serve as a solid foundation for further exploration and experimentation.

With the principles of transformers firmly established, we can now look forward to delving into more advanced topics and techniques in the next lecture. The knowledge and insights gained from this lecture will undoubtedly facilitate a deeper understanding of the complex mechanisms underlying large language models. As we continue to navigate the intricacies of building large language models from scratch, it's essential to remain engaged and motivated, and we look forward to seeing the progress and advancements that will be made in the subsequent lectures. Ultimately, the goal of building large language models from scratch is within reach, and with the knowledge of transformers, we are one step closer to achieving this ambitious objective.

Chapter 5

Lecture 5: How does GPT-3 really work?

The development of GPT-3 marks a significant milestone in the evolution of large language models, building upon the foundational concepts of Transformers and their progressive enhancements. This lecture will delve into the inner workings of GPT-3, tracing its roots back to the seminal "Attention is All You Need" paper in 2017, which introduced the revolutionary self-attention mechanism. By examining the advancements from Transformers to GPT, GPT-2, and ultimately GPT-3, we will gain a deeper understanding of the architecture and capabilities of this powerful model.

As we delve into the intricacies of building large language models from scratch, our focus shifts to one of the most significant advancements in the field: GPT, or generative pre-trained Transformer. To understand the evolution of GPT, it is essential to explore the progression of research papers that paved the way for its development. The journey begins with the introduction of Transformers, a paper released in 2017 titled "Attention is All You Need." This seminal work introduced the self-attention mechanism, a groundbreaking concept that enabled the capture of long-range dependencies in a sentence, thereby significantly improving the ability to predict the next word in a sequence. The original Transformer architecture, as proposed in the paper, consisted of an encoder and a decoder. However, the GPT architecture, which emerged subsequently, deviated from this design by relying solely on a decoder.

The introduction of the Transformer model marked a substantial shift in the landscape of natural language processing, surpassing the capabilities of recurrent neural networks and long short-term memory networks. The self-attention mechanism was the key innovation that allowed Transformers to excel in tasks that previously posed significant challenges. The GPT paper, which built upon the foundations laid by the Transformers paper, introduced the concept of generative pre-training. This approach was a significant departure from the traditional supervised learning methods that dominated the field of natural language processing at the time. The core idea behind generative pre-training is to leverage unsupervised learning to train a language model on a vast and diverse corpus of unlabeled text. The importance of this concept lies in the recognition that labeled data, which is essential for supervised learning, is often scarce and difficult to obtain, hindering the development of models that can perform adequately.

The notion of generative pre-training on unlabeled text is critical to understanding the GPT architecture. By pre-training a language model on a large corpus of text without labels, the model can learn to generate text that is coherent and contextually relevant. This approach enables the model to develop a deeper understanding of the structure and nuances of language, which is essential for tasks such as text generation and language translation. The GPT paper demonstrated that large gains in performance could be achieved by adopting this approach, and it paved the way for the development of subsequent models, including GPT-2, GPT-3, and GPT-4. As we explore the evolution of these models, we will also examine the concept of zero-shot versus few-shot learning, which is closely related to the idea of generative pre-training. By understanding the underlying principles of these models and their development, we can gain insight into the capabilities and limitations of large language models and the potential applications they enable.

The concept of generative pre-training, a crucial component of GPT-3, relies on two key elements: the use of unlabeled text and the process of next word prediction within that text. To understand this better, consider a sentence used as training data, where the next word prediction within that sentence serves as the testing data. This self-contained approach eliminates the need for labeled data, instead allowing the model to learn from the text itself. This idea of generative pre-training was first introduced in a 2018 paper, which built upon the Transformer architecture by removing the encoder block and focusing on unsupervised learning. By training a language model on a massive dataset to predict the next word, the authors demonstrated significant improvements in language understanding.

The 2018 paper marked the beginning of a new era in natural language processing, as it introduced the concept of generative pre-training and its potential to enhance language understanding. The authors' approach combined two essential ideas: the Transformer architecture, which had been introduced in a 2017 paper, and unsupervised pre-training, where labels are not provided and the model learns from the text itself. This innovative approach allowed the model to generate the next word in a sentence, hence the term "generative" pre-training. The fact that no pre-labeling of the data was required made this approach particularly appealing, as it enabled the model to learn from vast amounts of unlabeled text.

OpenAI's blog post on June 11, 2018, further highlighted the potential of this approach, showcasing state-of-the-art results on language tasks with a scalable system. The Transformer architecture and unsupervised pre-training formed the foundation of this new approach, which would later become a crucial component of GPT-3. Although this paper had a significant impact in the research community, with a large number of citations, it remained relatively unnoticed in the commercial sphere at the time. It wasn't until the following year, in 2019, that another paper built upon this concept and introduced the GPT-2 architecture, which would further push the boundaries of language understanding.

The GPT-2 paper introduced four different models, each with varying sizes, ranging from a smaller model to an extra-large model with 1.5 billion parameters. This marked the first time a paper had presented a large language model, and it paved the way for the development of even more advanced models. The release of GPT-2 small, medium, large, and extra-large models provided researchers with a range of options to explore the capabilities of generative pre-training. As we continue to explore the inner workings of GPT-3, it is essential to understand the foundation laid by these earlier papers and how they contributed to the development of this powerful language model. The next step in our journey will be to delve deeper into the GPT-2 architecture and its significance in the evolution of large language models.

The evolution of large language models has been a remarkable journey, marked by significant milestones that have transformed the field of natural language processing. One such milestone was the release of GPT-2, which introduced four different model sizes: small, medium, large, and extra-large. The GPT-2 extra-large model, in particular, was notable for its massive size, boasting 1 billion parameters, which led to impressive results at the time. This was a significant achievement, as it marked the first time a large language model of this scale had been developed, and it sparked widespread interest in the research community, as evidenced by the paper's impressive citation count of over 10,000.

The success of GPT-2 paved the way for even more advanced models, and in 2020, the GPT-3 model was introduced, which took the field by storm with its enormous size of 175 billion parameters. The GPT-3 model was released in various sizes, including small, medium, large, and versions with 2.7 billion and 6.7 billion parameters. However, it was the 175 billion parameter model that truly captured the imagination of researchers and the public alike, as it demonstrated an remarkable ability to perform a wide range of tasks, from translation and sentiment analysis to answering questions and recognizing emotions, despite being trained solely on the task of predicting the next word in a sequence. The sheer scale of GPT-3 was unprecedented, and it marked a major turning point in the development of large language models.

The introduction of GPT-3 was followed by the release of GPT-3.5, which became widely adopted and demonstrated the potential of these models in real-world applications. Today, we have GPT-4, which represents the latest iteration in this journey, and it is being used in a variety of contexts, including chat interfaces. Looking back, it is remarkable to see how far we have come in just seven years, from the original Transformer paper to the development of GPT, GPT-2, GPT-3, GPT-3.5, and finally, GPT-4. This rapid progress is a testament to the power of innovation and the dedication of researchers in the field. One common point of confusion is the difference between the Transformer architecture and GPT, with many people unaware that GPT borrows from the Transformer architecture but differs in its design, particularly in the absence of an encoder block.

As we delve deeper into the world of large language models, it is essential to understand the key concepts that underlie their behavior, including zero-shot and few-shot learning. Zero-shot learning refers to the ability of a model to generalize to completely unseen tasks without any prior specific examples, while few-shot learning involves learning from a minimal number of examples provided by the user. These concepts are crucial in understanding how models like GPT-3 can perform so many different tasks, despite being trained on a single objective. To gain a deeper understanding of these concepts, let us turn to the GPT-3 paper, which provides valuable insights into the design and capabilities of this groundbreaking model. By exploring the GPT-3 paper, we can gain a better understanding of the mechanisms that enable these models to perform so many different tasks, and how they have revolutionized the field of natural language processing.

As we delve into the inner workings of GPT-3, it's essential to understand the concept of few-shot learning, which enables the model to learn from a minimum number of examples provided as input. To gain a deeper understanding of this concept, let's turn to the GPT-3 paper, which illustrates zero-shot and few-shot learning through wonderful examples that clarify the concept. In zero-shot learning, the model predicts the answer given only a description, with no other assistance or support. For instance, if the prompt is to translate the word "cheese" from English to French, and the model is able to do so without any additional examples, that's an example of zero-shot learning.

In contrast, one-shot learning occurs when the model sees, in addition to the task description, a single example of the task. For example, if we tell the model to translate "cheese" into French, providing a single example, such as the translation of "otter" as a hint, that's one-shot learning. Few-shot learning takes this a step further, where the model sees a few examples of the task. For instance, if we provide the translations of "otter," "peppermint," and "giraffe" as supporting examples, and then ask the model to translate "cheese" into French, that's few-shot learning. These examples, provided in the GPT-3 paper, beautifully illustrate the differences between zero-shot, one-shot, and few-shot learning. Zero-shot learning requires no supporting examples, one-shot learning provides a single example, and few-shot learning provides a few examples.

The authors of the GPT-3 paper made a significant claim: they trained GPT-3, an auto-regressive language model with 175 billion parameters, 10 times more than any previous language model, and tested its performance in a few-shot setting. The results showed that GPT-3 achieved strong performance on tasks such as translation, question answering, and those that require on-the-fly reasoning or domain adaptation, like unscrambling words or performing three-digit arithmetic. This implies that GPT-3 is a few-shot learner, capable of performing tasks well when given a few examples, even though it was trained only for next-word prediction. The paper's claim that GPT-3 is a few-shot learner suggests that the model can learn to perform tasks with just a few examples, making it a powerful tool for a wide range of applications.

The implications of GPT-3's few-shot learning capabilities are significant, as they enable the model to adapt to new tasks and domains with minimal additional training. This raises important questions about the limitations and potential of few-shot learning, and how it can be applied in real-world scenarios. As we continue to explore the capabilities of GPT-3, we'll examine the potential applications of few-shot learning and the challenges that come with it. By understanding how GPT-3 learns from a few examples, we can unlock new possibilities for natural language processing and develop more advanced language models that can learn and adapt quickly.

In this chapter, we have explored the concept of few-shot learning and its significance in the context of GPT-3. We have seen how the model can learn from a few examples and perform tasks well, even though it was trained only for next-word prediction. In the next chapter, we will delve deeper into the architecture of GPT-3 and explore how its auto-regressive nature enables it to learn and generate text. We will also examine the challenges and limitations of training large language models like GPT-3 and discuss the potential applications of this technology.

As we delve into the inner workings of GPT-3, a crucial aspect of its functionality comes to the forefront: its ability to learn from a limited number of examples. A research paper on GPT-3 made a striking claim, asserting that this large language model is a "few-shot learner." In essence, this means that if you want GPT-3 to perform a specific task, such as language translation, you only need to provide it with a handful of examples to get it started. For instance, if you want GPT-3 to translate text from one language to another, you can simply give it a few examples of how certain phrases or sentences are translated, and it will then be able to perform the task on its own.

This concept of few-shot learning is an important one, as it highlights the model's ability to adapt and learn from a minimal amount of data. You may have encountered the terms "zero-shot" and "few-shot" learning in various books, articles, and blogs, and it's essential to understand the distinction between the two. Zero-shot learning refers to the model's ability to perform a task without any prior examples or context, whereas few-shot learning requires a small number of examples to get started. The question naturally arises: what about more advanced models like GPT-4? Is it a zero-shot learner, or does it still require a few examples to function effectively?

To answer this question, we can turn to GPT-4 itself and ask how it perceives its own learning abilities. When prompted, GPT-4 responds by saying that it is a few-shot learner, meaning that it can understand and perform tasks better with a few examples, although it can also handle many tasks without prior examples. This response is noteworthy, as it acknowledges the model's capability for zero-shot learning while also highlighting the benefits of providing a few examples to improve accuracy. In other words, GPT-4 can still generate responses without any context, but its performance improves significantly when given a few examples to work with.

This distinction is crucial to keep in mind when interacting with large language models like GPT-4. By providing a few examples of the desired output, you can significantly improve the model's responses. While GPT-4 does possess zero-shot capabilities, its few-shot learning abilities are more pronounced, and providing examples can help generate more accurate results. When asked if it also has zero-shot capabilities, GPT-4 confirms that it can perform tasks and answer questions without needing any prior examples or specific context. This underscores the model's versatility and ability to adapt to different situations.

In conclusion, GPT-4 embodies both zero-shot and few-shot learning abilities, but its performance is enhanced when provided with a few examples. This interplay between zero-shot and few-shot learning is essential to understand, as it can significantly impact the accuracy and effectiveness of large language models. As we look to the future and the development of even more advanced models like GPT-5 and GPT-6, it's likely that we'll see further improvements in zero-shot learning capabilities. For now, recognizing the distinction between zero-shot and few-shot learning can help you better utilize and interact with large language models, and we'll continue to explore the intricacies of these models in the chapters to come.

As we delve into the inner workings of GPT-3, it's essential to understand how this large language model handles various learning tasks, particularly zero-shot and few-shot learning. GPT-3 has shown impressive capabilities in zero-shot learning, where it can complete tasks without any examples. However, its performance can be further improved. To illustrate the difference between zero-shot and few-shot learning, consider a task where the input is "translate English to French." In zero-shot learning, the model would be given an input, such as the word "breakfast," and it would attempt to translate it without any prior examples. In contrast, few-shot learning provides the model with a few supporting examples to aid in the translation task.

For instance, if the task is to unscramble words or correct spellings, few-shot learning would involve providing the model with a few examples, such as a misspelled word like "goat" and its correct spelling. The model can then use these examples to learn the pattern and apply it to new, unseen inputs. In this case, if the model is given a misspelled word like "issue," it can use the provided examples to correctly translate it to "issue." This process demonstrates how few-shot learning enables the model to learn from a limited number of examples and apply that knowledge to new tasks.

The distinction between zero-shot and few-shot learning is crucial, as it highlights the model's ability to adapt to different learning scenarios. Zero-shot learning requires the model to rely solely on its prior knowledge and understanding of the task, whereas few-shot learning provides additional context and guidance through the supporting examples. By leveraging both zero-shot and few-shot learning, GPT-3 can develop a more comprehensive understanding of language and improve its performance on a wide range of tasks.

Another essential aspect of GPT-3's architecture is its utilization of large datasets. In the previous lecture, we touched upon the datasets used by GPT-3, but it's worth reiterating the significance of these datasets in the model's development. The common crawl dataset, in particular, plays a vital role in training GPT-3. This dataset is a free, open repository of web scrape data that has been maintained since 2007, containing over 250 billion pages spanning 17 years. GPT-3 uses 410 billion tokens from the common crawl dataset, which accounts for approximately 60% of the entire dataset. To develop an intuition for the scale of this dataset, it's helpful to think of one token as roughly equivalent to one word, although the actual process of tokenization is more complex. This massive dataset provides GPT-3 with an enormous amount of text data to learn from, which is essential for its language understanding capabilities.

In addition to the common crawl dataset, GPT-3 also utilizes data from web text, specifically the web text 2 corpus, which covers Reddit submissions from 2015 to 2020. This corpus consists of a vast amount of Reddit posts, providing the model with a diverse range of texts to learn from. By combining these large datasets, GPT-3 can develop a comprehensive understanding of language, enabling it to perform a wide range of tasks, from translation and text generation to question answering and more. As we continue to explore the architecture and capabilities of GPT-3, we'll delve deeper into the model's training process and the role of these large datasets in its development.

To comprehend the inner workings of GPT-3, it's essential to examine the vast amount of data on which it was trained. The model's training dataset, known as the web text, consists of a staggering 19 billion words, primarily comprising Reddit submissions from 2015 to 2020. This enormous corpus of text, which accounts for approximately 22% of the total data, is supplemented by a significant amount of text from books and Wikipedia, making up around 18-19% of the dataset. The sheer scale of this dataset becomes apparent when considering that the total number of tokens, or individual units of text, used for training GPT-3 is a staggering 300 billion. To put this into perspective, the enormity of this number is a testament to the immense computational resources required to process and train a model on such a vast amount of data.

The scale of the training data is a crucial aspect of GPT-3's development, as it necessitates a tremendous amount of computational power and, consequently, significant financial investment. The total pre-training cost for GPT-3 is estimated to be around \$4.6 million, a substantial expense that underscores the complexity and challenges involved in training a model of this magnitude. It's essential to recognize that pre-training is a critical phase in the development of GPT-3, as it lays the foundation for the model's subsequent fine-tuning and adaptation to specific tasks and domains. In the context of GPT-3, pre-training refers to the initial phase of training the model on a vast, unlabeled dataset, with the goal of developing a broad understanding of language and text structures.

As we delve deeper into the workings of GPT-3, it's worth noting that the pre-trained model is often referred to as the base model or foundational model, which can be fine-tuned later for specific applications. Fine-tuning involves adapting the pre-trained model to a particular task or domain, using a smaller amount of labeled data. This process is crucial for achieving optimal performance in real-world settings, such as when a company wants to utilize GPT-3 for generating text specific to their industry or application. For instance, an educational company seeking to use GPT-3 to generate multiple-choice questions would need to fine-tune the model on their proprietary dataset to ensure the output is relevant and accurate. The distinction between pre-training and fine-tuning is vital, as it highlights the different phases of the model's development and the varying requirements for each phase.

The concept of tokenization, which refers to the process of breaking down text into individual units or tokens, is also essential to understanding how GPT-3 processes and generates text. While we can think of a token as equivalent to a word for now, this concept will be explored in greater detail in subsequent discussions on tokenization. The main paper on GPT-3, titled "Language Models are Few-Shot Learners," provides valuable insights into the model's architecture and training process, and serves as a foundation for further exploration of the model's capabilities and limitations. As we continue to explore the intricacies of GPT-3, it's clear that the model's pre-training phase is a critical component of its development, and that the subsequent fine-tuning phase is essential for adapting the model to specific tasks and domains.

As we delve into the inner workings of GPT-3, it becomes clear that fine-tuning is a crucial step in achieving robust and reliable outcomes, especially in production-level settings. For instance, an educational company seeking to generate multiple-choice questions can leverage GPT-3 or its successor, but to ensure optimal results, fine-tuning on a labeled dataset collected over several years is essential. This process allows the model to adapt to the company's specific needs and improve its performance on the task at hand.

While many pre-trained language models, including GPT-3 and GPT-4, are available as open-source models, they can be used as general-purpose tools for tasks such as writing, extracting, and editing text, even without fine-tuning. This makes them highly accessible for individuals, including students, who can utilize these models for various purposes, such as analyzing PDFs or gathering information. However, it is essential to recognize the distinction between open-source and closed-source language models. Closed-source models, like GPT-4, conceal their parameters and weights, limiting users to interacting with the model through a predefined interface. In contrast, open-source models provide transparency into their internal workings, allowing for greater customization and adaptability.

A comparison of the performance of open-source and closed-source models reveals an intriguing trend. In 2022, closed-source models, such as GPT-4, outperformed their

open-source counterparts. However, as time progressed, the performance gap between the two narrowed, with open-source models like LLaMA 3.1, released by Meta, beginning to rival the capabilities of closed-source models. LLaMA 3.1, with its 400 billion parameters, has demonstrated performance comparable to, if not surpassing, GPT-4. This convergence of open-source and closed-source model performance is a significant development, as it suggests that the benefits of large language models can be more widely accessible, even to those without access to proprietary models.

For students and individuals who do not require customized solutions, interacting with pre-trained models like GPT-3 and GPT-4 can be a seamless and effective way to leverage the capabilities of large language models. Without the need to fine-tune or access the models' internal parameters, users can still reap the benefits of these powerful tools. Having explored the pre-training and fine-tuning processes, as well as the open-source and closed-source ecosystem, we can now shift our focus to the architecture underlying GPT-3. The GPT architecture, as we have seen in previous discussions, consists solely of a decoder block, differing from the original Transformer model, which comprises both encoder and decoder components. This distinctive design is a key factor in GPT-3's ability to generate coherent and contextually relevant text, and understanding its architecture is essential for appreciating the model's capabilities and limitations.

GPT-3 is a scaled-up version of the original Transformer model, which was implemented on a larger dataset. The original Transformer consists of an encoder and a decoder, whereas GPT-3 is a decoder-only model, as we have previously discussed. The GPT-3 model is also a scaled-up version of the model introduced in the 2018 paper on generative pre-training, with around 175 billion parameters. This significant increase in parameters allows GPT-3 to process and generate vast amounts of text.

One of the key aspects of GPT models is that they are trained on next-word prediction tasks, which means that given a sequence of words, the model predicts the next word in the sequence. For example, if the input is "the lion," the model would predict that the next word is likely to be "jungle." This training process is crucial to understanding how GPT models work, and it is essential to examine it in more detail. The training process involves breaking up a sentence into input and output, where the input consists of a few words, and the output is always one word. The model is then trained to predict the next word in the sequence, given the input.

To illustrate this process, consider an example where the input is "second law of Robotics: a," and the model is trained to predict the next word. The output of the previous step becomes the input for the next step, so the input would be "second law of Robotics: a robot," and the model would predict the next word. This process continues, with the model predicting the next word in the sequence at each step. What is notable about this training process is that it does not involve labeled data, where the correct output is provided. Instead, the sentence itself provides the labels, as the model is trained to predict the next

word in the sequence. This is why GPT models are considered unsupervised learning models, as they do not require labeled data to learn.

The fact that GPT models are trained only on next-word prediction tasks is quite remarkable, given that they can still perform a wide range of other tasks, such as translation and spelling correction. However, this training process requires a huge amount of compute power, which is why training a model like GPT-3 can be so expensive. The costs involved in training such a model are significant, with estimates suggesting that it can cost millions of dollars to train a single model. This is due to the vast amounts of data that need to be processed, which requires significant computational resources. As we continue to explore the inner workings of GPT-3, it becomes clear that the model's ability to predict the next word in a sequence is just the beginning of its capabilities.

The training process of GPT-3 is a complex and computationally intensive task that requires a deep understanding of the underlying mechanics of the model. As we have seen, the model is trained on next-word prediction tasks, which allows it to learn the patterns and structures of language. This training process is essential to the model's ability to generate coherent and contextually relevant text. In the next part of our discussion, we will delve deeper into the implications of this training process and explore how it enables GPT-3 to perform a wide range of tasks. We will also examine the limitations of this approach and discuss potential avenues for further research and development.

The sheer scale of the task of training a model like GPT-3 is daunting, requiring a massive amount of compute power - to the tune of \$4.6 million - due to the enormous amounts of data involved. To put this into perspective, consider a dataset consisting of 410 billion words, which can be roughly estimated to contain around 40 billion sentences. Each of these sentences must be broken down into input and output pairs, with the output requiring the prediction of the next word. This process is highly time-consuming, as it needs to be repeated for billions of sentences, thereby necessitating enormous computational resources for the training procedure.

The training process itself is an iterative one, where the model initially predicts incorrect outputs, but is then corrected using the actual output, which is known from the sentence itself. The error between the predicted output and the correct output is computed, and the weights of the Transformer or GPT architecture are adapted using a process similar to backpropagation in neural networks, allowing the model to learn from its mistakes and improve its predictions over time. This self-supervised learning approach allows the model to learn from the data itself, without the need for external labeling.

In the case of GPT-3, this self-supervised learning process is exemplified by the way a sentence is divided into training and testing pairs. The true next word in a sentence is known, and this is used as the label to train the model. The model attempts to predict the next word, and the difference between the predicted word and the true word is used to

update the weights of the neural network. This process is repeated for the entire dataset, allowing the model to learn the patterns and relationships within the data. The fact that the model can learn from the data itself, without the need for external labeling, is what makes this approach so powerful, and is a key factor in the success of GPT-3.

The use of the previous output as the input for future predictions is another key aspect of the auto-regressive model, and is what gives rise to its name. This process allows the model to build on its previous predictions, using the output of one sentence as the input for the next. For example, if the model is given the sentence "Second law of Robotics", the output might be the word "U", which then becomes the input for the next sentence, allowing the model to generate text that is coherent and context-dependent. This auto-regressive property of the model is what enables it to generate long sequences of text that are often highly realistic and engaging. As we continue to explore the workings of GPT-3, we will delve deeper into the implications of this approach, and examine the ways in which it can be used to generate high-quality text.

As we delve into the inner workings of GPT-3, it becomes clear that the model's architecture is designed to facilitate a specific type of learning. The output of one iteration becomes the input to the next, creating a continuous loop of word prediction. For instance, if the input is "second law of Robotics: U", the model predicts the next word to be "robot", which then becomes the input to the next sentence. This process is a hallmark of autoregressive models, where previous outputs are used as inputs for future predictions.

The pre-training phase of GPT models is a crucial aspect of their development, and it is worth noting that this phase is unsupervised. The structure of the data itself is used to create the labels, where the next word in a sentence serves as the label. This approach allows the model to learn from the data without requiring explicit supervision. The autoregressive nature of the model is also essential, as it enables the model to generate text that is coherent and context-dependent.

The GPT architecture is simpler than the original Transformer architecture, as it only consists of the decoder block, omitting the encoder block. To illustrate this, consider the original Transformer architecture, which comprises both an encoder and a decoder. In contrast, the GPT architecture only passes the input text to the decoder, eliminating the need for an encoder. While the GPT architecture may be simpler, it is by no means less complex, as it consists of a massive 96 Transformer layers and 175 billion parameters. To put this into perspective, the original Transformer had six encoder-decoder blocks, whereas the GPT-3 architecture uses 96 Transformer layers, resulting in an enormous number of parameters.

The sheer number of parameters in the GPT-3 architecture is a key factor in its ability to predict the next word in a sentence with high accuracy. However, this also means that the pre-training phase requires significant computational resources, as the model must

optimize 175 billion parameters to achieve optimal performance. The process of predicting the next word in a sentence is a complex one, involving multiple iterations and the use of previous outputs as inputs for future predictions. By understanding how the GPT model works, we can appreciate the complexity and sophistication of this technology.

To further illustrate the process of next word prediction, consider the visual representation of the GPT architecture. The model consists of multiple iterations, each of which involves the prediction of the next word in a sentence. By zooming in on a single iteration, we can see how the model uses the previous output as input to the next iteration, creating a continuous loop of word prediction. This process is essential to the model's ability to generate coherent and context-dependent text. As we continue to explore the GPT-3 architecture, we will delve deeper into the intricacies of this process and examine the implications of this technology for natural language processing and generation.

The architecture of GPT-3 is a complex interplay of iterative predictions, where each output informs the input of the next iteration. To understand this process, let's delve into the individual iterations that comprise the GPT-3 model. We can start by examining iteration number one, where the input consists of a single word. This word undergoes pre-processing, which involves converting it into token IDs. The token IDs are then passed through the decoder block, followed by the output layer, where the next word is predicted. In this case, the predicted next word is "is", which becomes part of the input for the second iteration.

As we move to iteration two, the input now includes the previously predicted word "is", along with the original input word "this". The same steps are repeated: token ID pre-processing, followed by the decoder block and output layer, resulting in the prediction of the next word, which is "so". This output from iteration two becomes the input for iteration three, where the process is repeated once more. The output from each iteration informs the input of the next iteration, demonstrating the auto-regressive nature of the GPT-3 model. This iterative process continues, with each prediction building upon the previous one, allowing the model to generate coherent and context-dependent text.

The GPT architecture is characterized by the absence of an encoder block, with only the decoder block present. This is evident in the iterative process, where the output from each iteration is used as input for the next, without the need for an encoder to process the input. The decoder block is responsible for generating the next word in the sequence, based on the input it receives from the previous iteration. This design enables the GPT-3 model to learn complex patterns and relationships in language, allowing it to generate high-quality text that is often indistinguishable from human-written content.

One of the key benefits of the GPT-3 architecture is the emergence of unexpected capabilities, despite being trained only for next-word prediction. This phenomenon is known as emergent behavior, where the model develops additional abilities beyond its

primary objective. For example, GPT-3 can perform language translation tasks, such as converting the word "breakfast" into French, despite not being explicitly trained for this task. This emergent behavior is a result of the model's ability to learn and represent complex patterns in language, allowing it to generalize and adapt to new tasks and contexts. The emergence of these capabilities is a testament to the power and flexibility of the GPT-3 architecture, and highlights the potential for large language models to be used in a wide range of applications.

As we conclude our exploration of the GPT-3 architecture, it is clear that the iterative and auto-regressive nature of the model plays a crucial role in its ability to generate high-quality text. The emergence of unexpected capabilities, such as language translation, demonstrates the potential for large language models to be used in a variety of applications, from text generation to language understanding. In the next chapter, we will delve deeper into the training process of GPT-3, exploring the techniques and strategies used to optimize the model's performance and enable it to learn from large datasets. We will also examine the challenges and limitations of training large language models, and discuss the potential implications of these models for the field of natural language processing.

As we delve into the intricacies of GPT-3, it becomes apparent that this language model has developed a range of capabilities that extend far beyond its initial training objectives. This phenomenon is known as emergent behavior, where the model demonstrates an ability to perform tasks that it was not explicitly trained to accomplish. A notable example of this is GPT-3's capacity for translation tasks, despite not being specifically trained for this purpose. Furthermore, the model can also generate multiple-choice questions, as demonstrated by the MCQ generator, where it can produce questions on a given topic, such as gravity, without having been explicitly trained to do so.

The emergence of these capabilities is a direct result of the model's pre-training process, which enables it to predict the next word in a sequence. This ability to predict the next word allows the model to develop a deeper understanding of language and its nuances, ultimately giving rise to emergent behavior. The implications of this are profound, as GPT-3 can perform a wide range of tasks, including answering text questions, generating worksheet sheets, summarizing text, creating lesson plans, and even generating report cards and grade essays. This is a testament to the model's ability to generalize and adapt to new tasks, even when it has not been explicitly trained on them.

The concept of emergent behavior is not unique to GPT-3, but it is a characteristic that has been observed in other large language models as well. As noted in a blog post by OpenAI, the underlying language model can be used to perform tasks without ever having been trained on them, and the performance on these tasks improves as the underlying language model improves. This is a clear example of emergent behavior, where the model is able to develop new capabilities without explicit training. The formal definition of emergent

behavior refers to the ability of a model to perform tasks that it was not explicitly trained to perform, and this phenomenon remains an open question in the field of natural language processing.

The study of emergent behavior is an active area of research, with many papers exploring its implications and applications. A search on Google Scholar yields a multitude of results, including papers on exploring emergent behavior in large language models. This is an area that holds great promise for future research, and it is an exciting topic for those looking to contribute to the field. As we conclude this lecture, we can reflect on the many topics that we have covered, from the history of attention mechanisms to the concepts of zero-shot and few-shot learning. Our exploration of GPT-3 has revealed the impressive capabilities of this language model, and we have gained a deeper understanding of how it works. In the next chapter, we will continue to explore the intricacies of large language models, and we will delve into new topics that will further our understanding of this complex and fascinating field.

To understand how GPT-3 works, it's essential to delve into the history of large language models, which began with the introduction of the Transformer architecture in the 2017 paper "Attention is All You Need". This groundbreaking work laid the foundation for subsequent developments, including the creation of Generative Pre-Training (GPT), which modified the Transformer architecture by utilizing only a decoder and no encoder. GPT introduced two key concepts: unsupervised learning and auto-regressive processing of unlabelled data, which eliminated the need for labelled data during pre-training. The GPT architecture evolved over time, with the release of GPT-2 in 2019, which came in four models with varying parameter sizes, ranging from 117 million to 1.5 billion parameters.

The next significant milestone was the release of GPT-3 in 2020, which boasted an unprecedented 175 billion parameters, making it the largest language model at the time. This model was particularly notable for its ability to perform remarkably well on few-shot learning tasks, where the model is provided with a limited amount of supplementary data to learn from. In fact, the authors of the GPT-3 paper explicitly stated that language models are few-shot learners, implying that they excel in situations where some examples are provided. Interestingly, while GPT-3 is capable of zero-shot learning, where no examples are given, its performance is substantially better when provided with a few examples, making it a more accurate few-shot learner.

The vast amount of data used to train GPT-3 is also noteworthy, with the model being trained on approximately 300 billion tokens, or roughly 300 billion words. To put this into perspective, a token represents a unit of text that the model processes, which can be thought of as a single word for simplicity. The enormity of this dataset is a significant factor in the model's impressive capabilities. However, this massive dataset comes at a cost, with the pre-training process requiring a staggering \$4.6 million. The reason for this substantial expense lies in the auto-regressive nature of the model, which involves predicting the next

word in a sentence. This process requires breaking down sentences into training and testing data, using one word for testing and the remaining words for training, a procedure that must be repeated for all the sentences in the vast dataset, resulting in a substantial computational cost.

The complexity of training GPT-3 is further exacerbated by the sheer number of parameters, with 175 billion parameters requiring optimization, a process that demands significant computational resources. This highlights the challenges involved in developing and training large language models like GPT-3, which push the boundaries of what is possible in natural language processing. As we continue to explore the capabilities and limitations of GPT-3, it is essential to appreciate the historical context and the significant advancements that have led to the development of this powerful language model.

In conclusion, this chapter has provided a historical context for the development of GPT-3, highlighting its evolution from earlier models and its ability to perform impressively on few-shot learning tasks. The next chapter will delve deeper into the technical details of GPT-3, exploring its architecture, training process, and the implications of its massive parameter size, providing a more comprehensive understanding of this powerful language model.

As we delve deeper into the intricacies of GPT-3, it becomes clear that its sheer scale is a significant factor in its impressive capabilities. With millions of data files at its disposal, the model requires an enormous amount of computational power to optimize its 175 billion parameters. This is why training GPT-3 is such a daunting task, necessitating access to high-performance GPUs and other specialized hardware. The GPT architecture itself is noteworthy, comprising only a decoder that operates in an iterative, auto-regressive manner. In each iteration, the output from the previous step is fed back in as input, allowing the model to generate text based on its own previous outputs. This process is rooted in unsupervised learning, where the model is trained to predict the next word in a sentence without explicit supervision.

The pre-training process is a critical component of GPT-3's development, and it is often followed by a fine-tuning stage where the model is trained on a narrower, more specific dataset to enhance its performance on production-level tasks. This two-stage approach has become a standard practice in the development of large language models. Interestingly, the gap between open-source and closed-source language models is beginning to narrow, thanks in part to the introduction of models like LLaMA 3.1, which boasts an impressive 405 billion parameters and has been shown to outperform GPT-4 in certain benchmarks.

One of the most fascinating aspects of large language models like GPT-3 is their ability to exhibit emergent behavior. This refers to the model's capacity to perform tasks that it was not explicitly trained to accomplish, such as generating multiple-choice questions, creating

lesson plans, or even translating text. Despite being trained solely on next-word prediction, GPT-3 is able to generalize to a wide range of tasks, making it an incredibly versatile tool. The study of emergent behavior is an active area of research, and it holds tremendous potential for future innovations in the field of natural language processing.

As we conclude our exploration of how GPT-3 works, it is clear that the model's impressive capabilities are rooted in its complex architecture and massive scale. Over the course of the past five lectures, we have covered the fundamentals of large language models, from the basics of transformer architectures to the intricacies of pre-training and fine-tuning. In the next chapter, we will shift our focus to the practical aspects of building a large language model from scratch, beginning with the critical stage of data pre-processing. This will mark the beginning of a new phase in our journey, where we will delve into the hands-on aspects of model development, and we will explore the many challenges and opportunities that arise when working with these complex systems.

Glossary

AI: Artificial intelligence refers to the development of computer systems that can perform tasks that typically require human intelligence, such as understanding language, recognizing images, and making decisions. In the context of large language models, AI is used to enable machines to understand and generate human-like language.

Chatbot: A chatbot is a computer program designed to simulate conversation with human users, either through text or voice interactions. Examples of chatbots include Eliza and Chat GPT, which demonstrate the progression of natural language processing capabilities.

Closed-Source Model: A closed-source model refers to a type of artificial intelligence model where the architecture and weights of the model are not publicly available, restricting access to the model's internal workings. Examples of closed-source models include those released by Open AI.

Deep Learning: Deep learning is a subfield of machine learning that involves the use of neural networks to analyze and interpret data, such as images, speech, and text. In the context of large language models, deep learning is used to enable machines to learn and generate human-like language.

Eliza: Eliza is an early chatbot developed in the 1960s, designed to simulate a therapist by responding to user input with simplistic and vague responses. Eliza represents one of the first attempts at creating a natural language processing system.

Generative AI: Generative AI refers to a type of artificial intelligence that enables machines to generate new content, such as text, images, videos, and audio, based on patterns and structures learned from existing data. Large language models are a key application of generative AI.

GP4: GP4, also known as GPT-4, is a closed-source large language model released in 2023, which demonstrated impressive functionality and had a significant impact on the field of natural language processing.

Large Language Model (LLM): A large language model is a type of artificial intelligence model designed to process and generate human-like language, typically using deep learning techniques and large amounts of training data. LLMs have the ability to understand and respond to complex user input.

LLaMA 3.1: LLaMA 3.1 is an open-source large language model released by Facebook, which performs on par with closed-source models like GP4. LLaMA 3.1 demonstrates the

advancements in open-source models and the narrowing gap between open-source and closed-source models.

Machine Learning: Machine learning is a subfield of artificial intelligence that involves the development of algorithms and statistical models that enable machines to learn from data, without being explicitly programmed. Machine learning is a key component of large language models.

Natural Language Processing (NLP): Natural language processing refers to the field of study focused on enabling machines to understand, interpret, and generate human language, including tasks such as language translation, text summarization, and sentiment analysis.

Neural Network: A neural network is a type of machine learning model inspired by the structure and function of the human brain, consisting of layers of interconnected nodes (neurons) that process and transmit information. Neural networks are a key component of deep learning and large language models.

Open-Source Model: An open-source model refers to a type of artificial intelligence model where the architecture and weights of the model are publicly available, allowing anyone to access and learn from the model's internal workings. Examples of open-source models include LLaMA 3.1.

Text Generation: Text generation refers to the ability of machines to create human-like text based on patterns and structures learned from existing data. Large language models are capable of generating coherent and context-specific text.

Video Generation: Video generation refers to the ability of machines to create realistic videos based on patterns and structures learned from existing data. Generative AI has enabled the creation of stunningly realistic AI-generated videos.

YouTube Channel (VIUA): VIUA is a YouTube channel created by Dr. Raj Dander, aiming to make AI accessible to everyone by providing comprehensive guides to building large language models from scratch, without assuming prior knowledge.